Deloitte.

# Introduction to ABAP Objects

Deloitte Technology Academy (DTA)

# Agenda

| Topic | Descriptions | Duration |
|---|---|---|
| Overview of Object Oriented Programming in ABAP | Why Object Orientation?, Benefits of ABAP Objects, and Principal Features of OO-ABAP | 8 hours |
| Classes in ABAP | Components of Class, Constructors, Declaring and Using a Global Class, When to Use Local Classes, Events, and Interfaces | |
| Inheritance | Inheritance Overview, Single Inheritance, Redefining Methods, and Constructors | |
| Error Handling | Class Exceptions, and Class Debugging | |

Copyright © 2022 Deloitte Development LLC. All rights reserved.

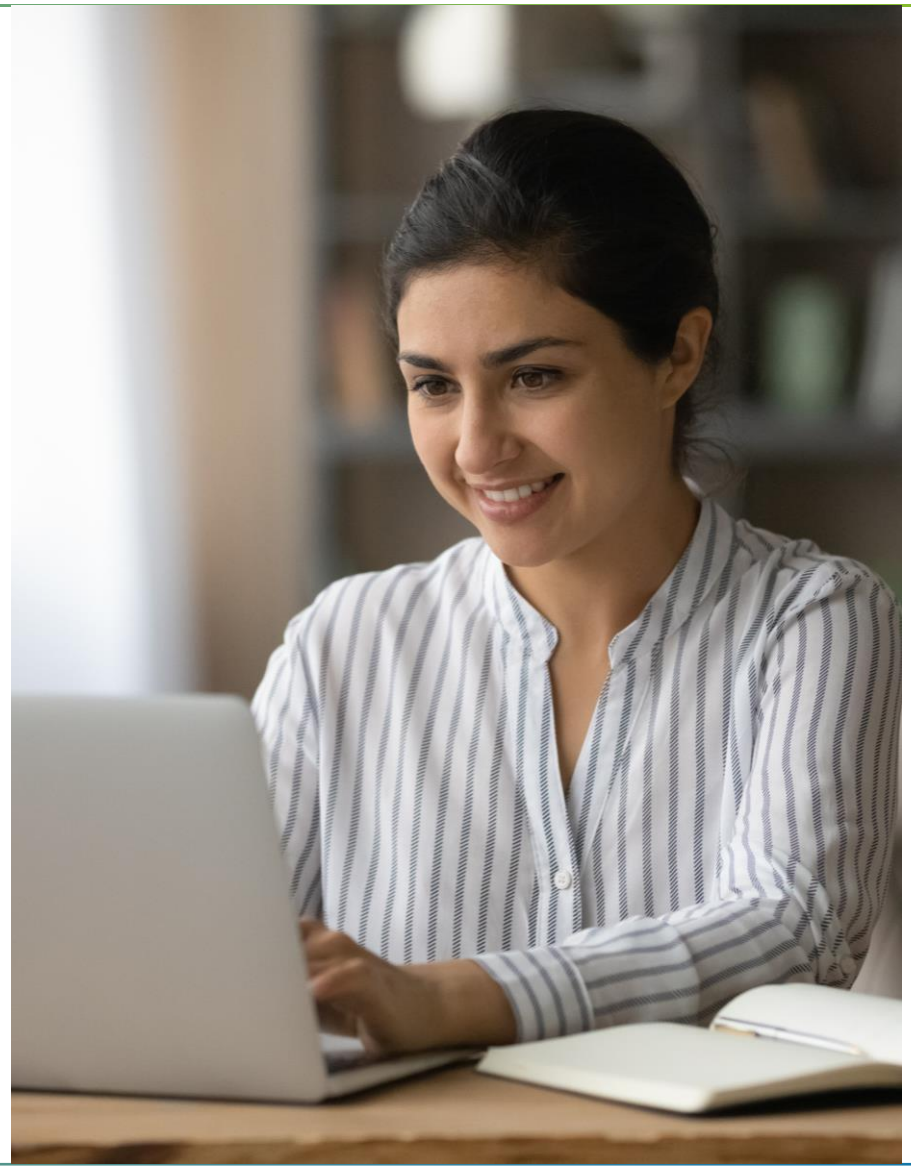
# Learning Objectives

By the end of this session, you will be able to:

- Describe the features of Object-Oriented Programming (OOP)
- Create local and global classes in ABAP
- Create class components like attributes, and methods and deploy them in ABAP
- Create constructors and applications in ABAP
- Explain the need for static components in ABAP
- Describe how to implement static components in ABAP
- Implement event handling in ABAP
- Create and use an interface
- Discuss the concept of inheritance
- Explain runtime polymorphism
- Handle class-based exceptions

Copyright © 2022 Deloitte Development LLC. All rights reserved.

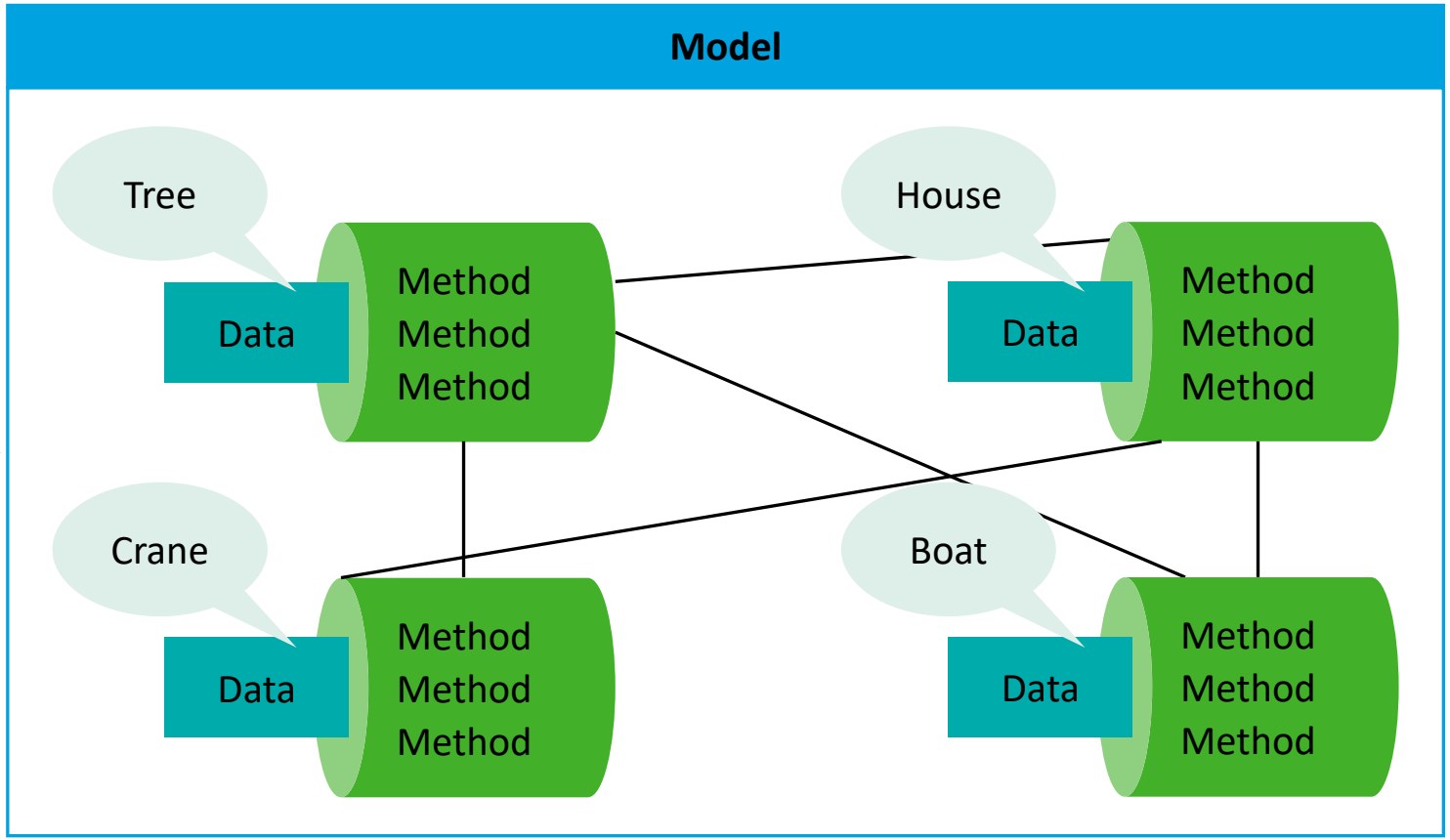
# Overview of Object-Oriented Programming in ABAP



# What Are Objects?

- Objects are an abstraction of the real world.
- Objects are units made of data, and of the functions belonging to that data.

**Real world**






Copyright © 2022 Deloitte Development LLC. All rights reserved.

# What Are Objects? (Cont.)

- An object is a thing, an entity, a noun, something you can pick up or kick, or anything you can imagine that has its own identity.
- Some objects are living, and some aren't. Examples from the real world include a car, a person, a house, a table, a dog, a pot plant, a checkbook, or a raincoat.
- Real-world objects share two characteristics: They all have attributes and behavior.

A Person

A Date

A Cat

A Bank Account

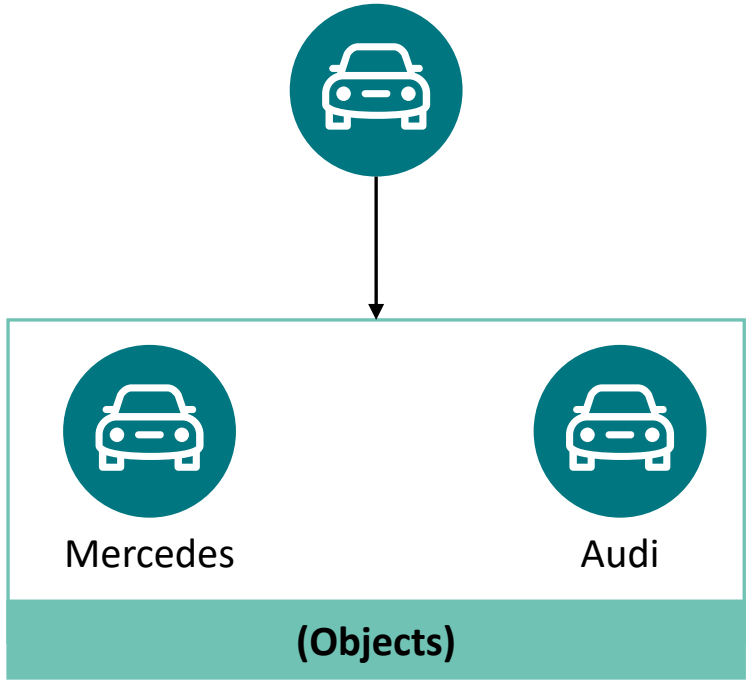
A Number



Copyright © 2022 Deloitte Development LLC. All rights reserved.

# Depicting an Object—Example

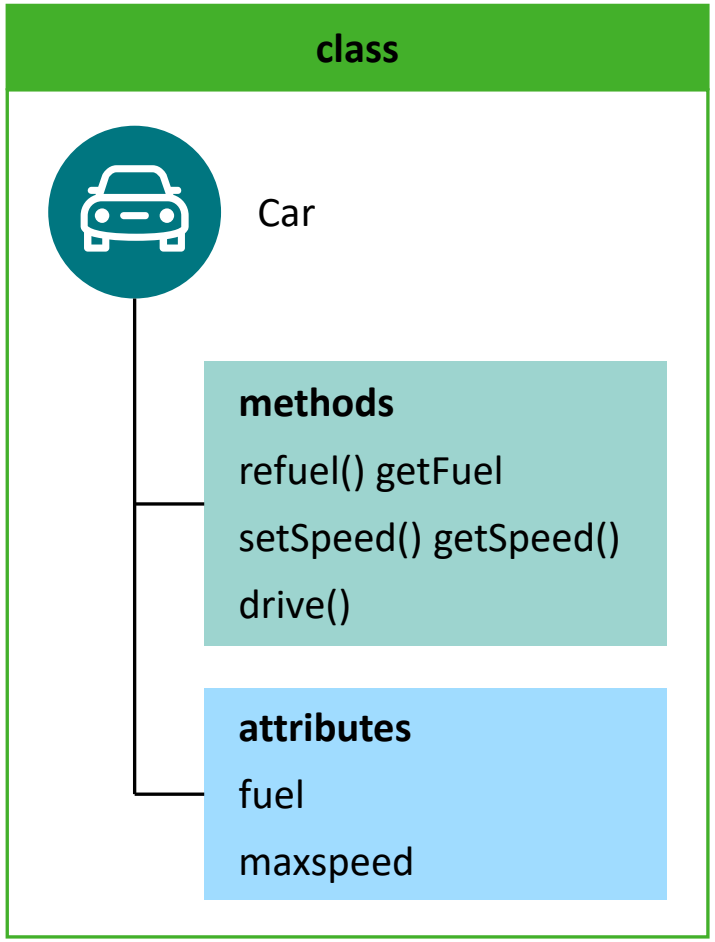
- Car is a class
- Audi, Mercedes, etc., are Objects of the class named “Car”

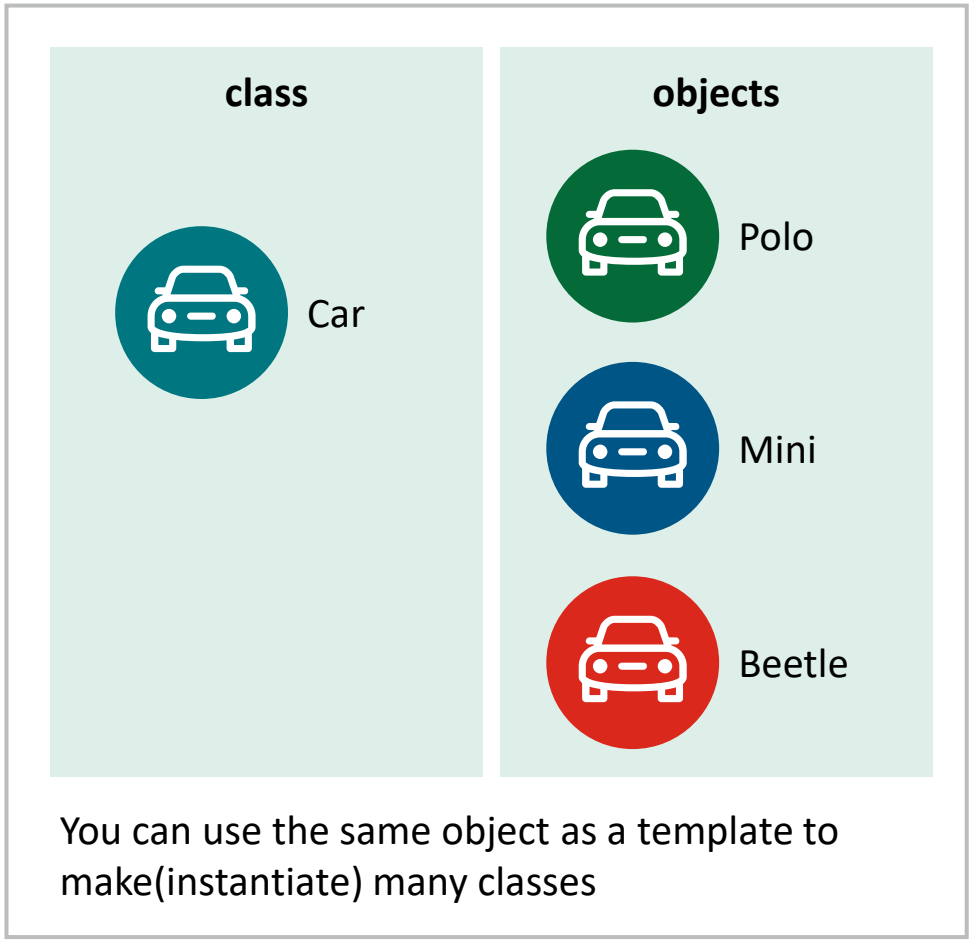





Copyright © 2022 Deloitte Development LLC. All rights reserved.

# Classes and Objects




You can use the same object as a template to make(instantiate) many classes

Copyright © 2022 Deloitte Development LLC. All rights reserved.

# Basic Building Blocks of OOP

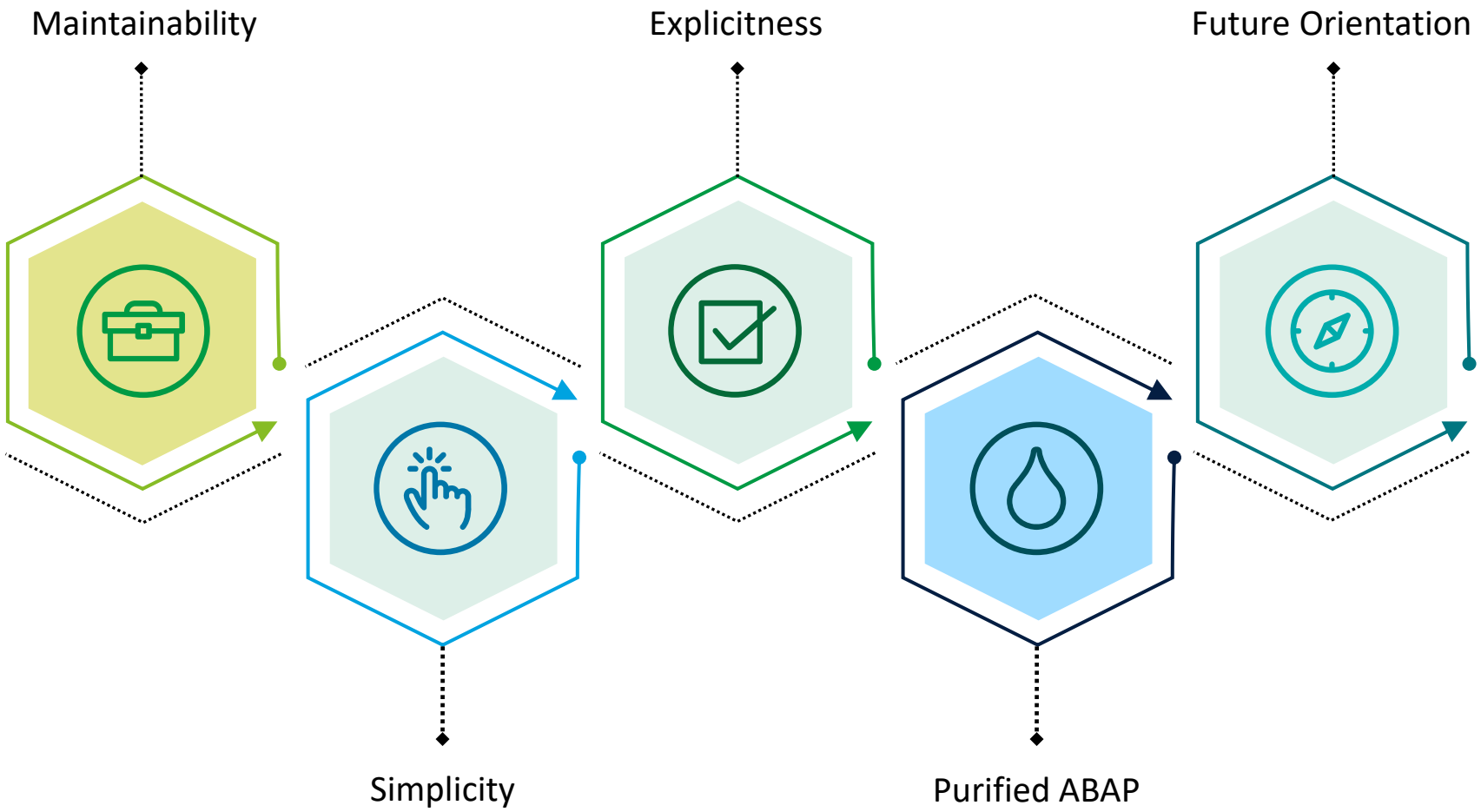
- **Object-Oriented Programming (OOP)** is a programming methodology using objects consisting of data and methods.
- Classes and objects are the basic building blocks of Object-Oriented Programming.

| | |
|---|---|
| **Classes** | When a real-world entity is modeled into the OOP world, then it is known as Class, characteristics as attributes, and functionality as methods. |
| **Objects** | Objects are an instance of a Class. |



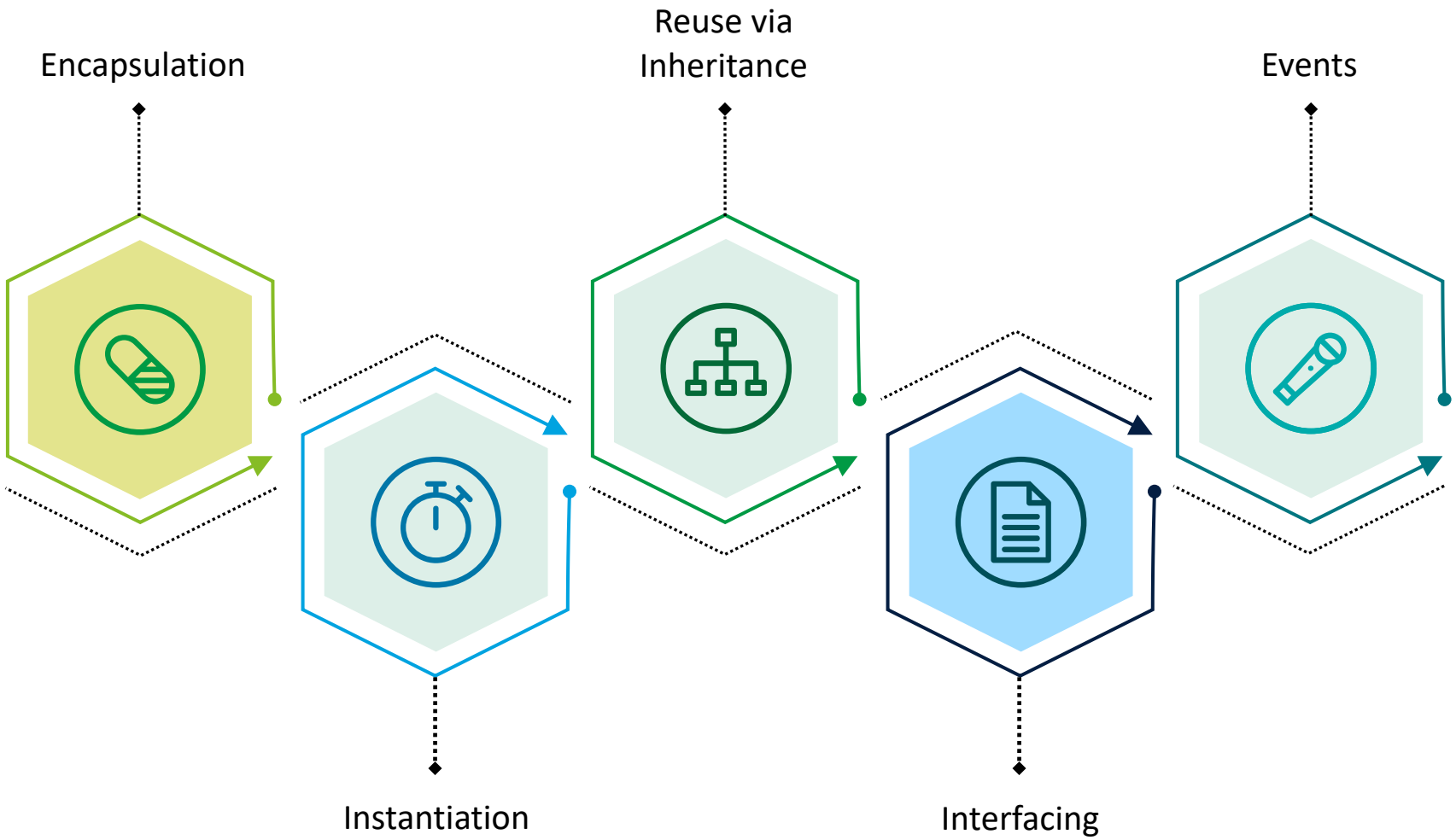
Copyright © 2022 Deloitte Development LLC. All rights reserved.

# Benefits of ABAP Objects




Copyright © 2022 Deloitte Development LLC. All rights reserved.

# Principal Features of ABAP Objects

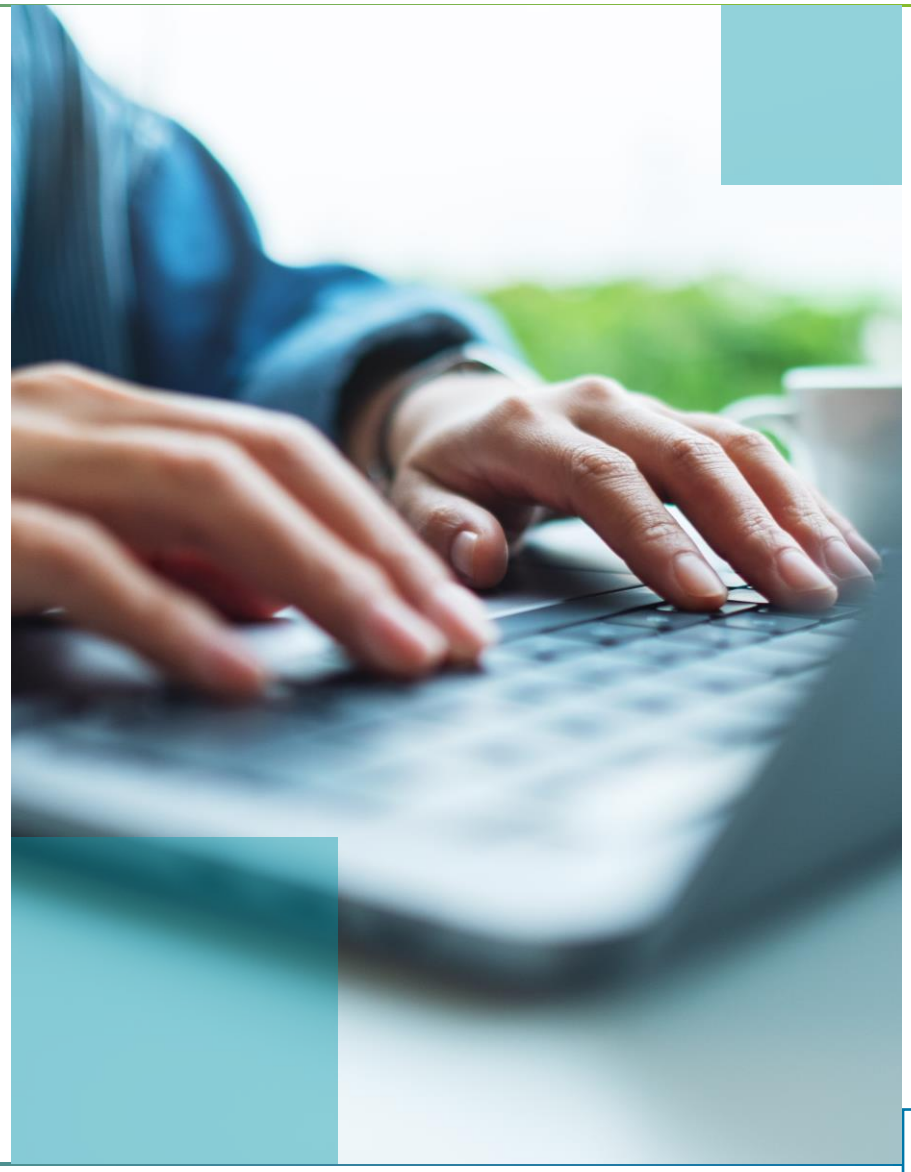





Copyright © 2022 Deloitte Development LLC. All rights reserved.

# Best Practices of OOP in ABAP

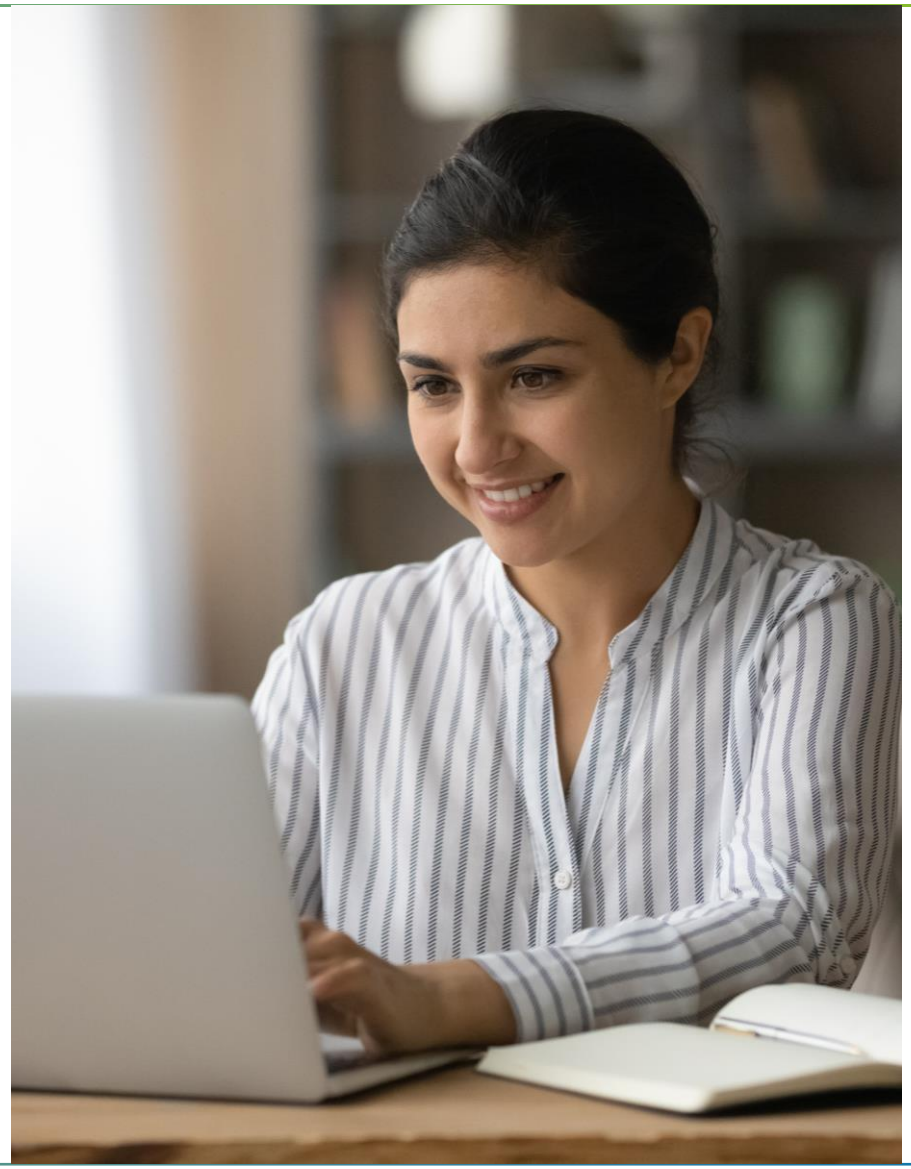
- Use ABAP objects for new and ongoing projects
- Take advantage of object-oriented features
- Use methods as far as possible, even if you stay within the procedural programming model
- Do not use subroutines anymore
- Use function modules only when technically necessary (RFC, encapsulation of screens, etc.)
- Disentangle classical ABAP from ABAP Objects



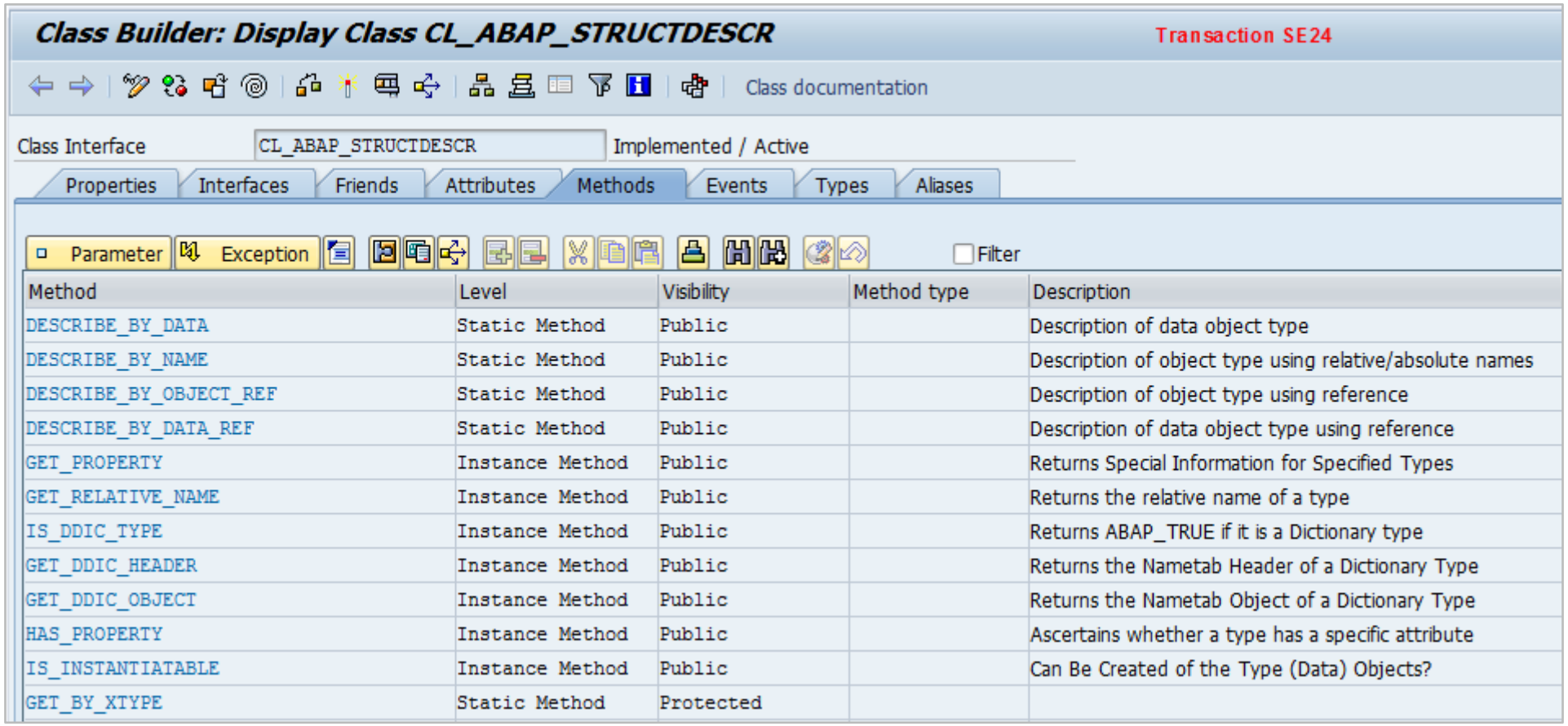


Copyright © 2022 Deloitte Development LLC. All rights reserved.

# Classes in ABAP



# Class in SAP




| Method | Level | Visibility | Method type | Description |
|---|---|---|---|---|
| DESCRIBE_BY_DATA | Static Method | Public | | Description of data object type |
| DESCRIBE_BY_NAME | Static Method | Public | | Description of object type using relative/absolute names |
| DESCRIBE_BY_OBJECT_REF | Static Method | Public | | Description of object type using reference |
| DESCRIBE_BY_DATA_REF | Static Method | Public | | Description of data object type using reference |
| GET_PROPERTY | Instance Method | Public | | Returns Special Information for Specified Types |
| GET_RELATIVE_NAME | Instance Method | Public | | Returns the relative name of a type |
| IS_DDIC_TYPE | Instance Method | Public | | Returns ABAP_TRUE if it is a Dictionary type |
| GET_DDIC_HEADER | Instance Method | Public | | Returns the Nametab Header of a Dictionary Type |
| GET_DDIC_OBJECT | Instance Method | Public | | Returns the Nametab Object of a Dictionary Type |
| HAS_PROPERTY | Instance Method | Public | | Ascertains whether a type has a specific attribute |
| IS_INSTANTIATABLE | Instance Method | Public | | Can Be Created of the Type (Data) Objects? |
| GET_BY_XTYPE | Static Method | Protected | | |

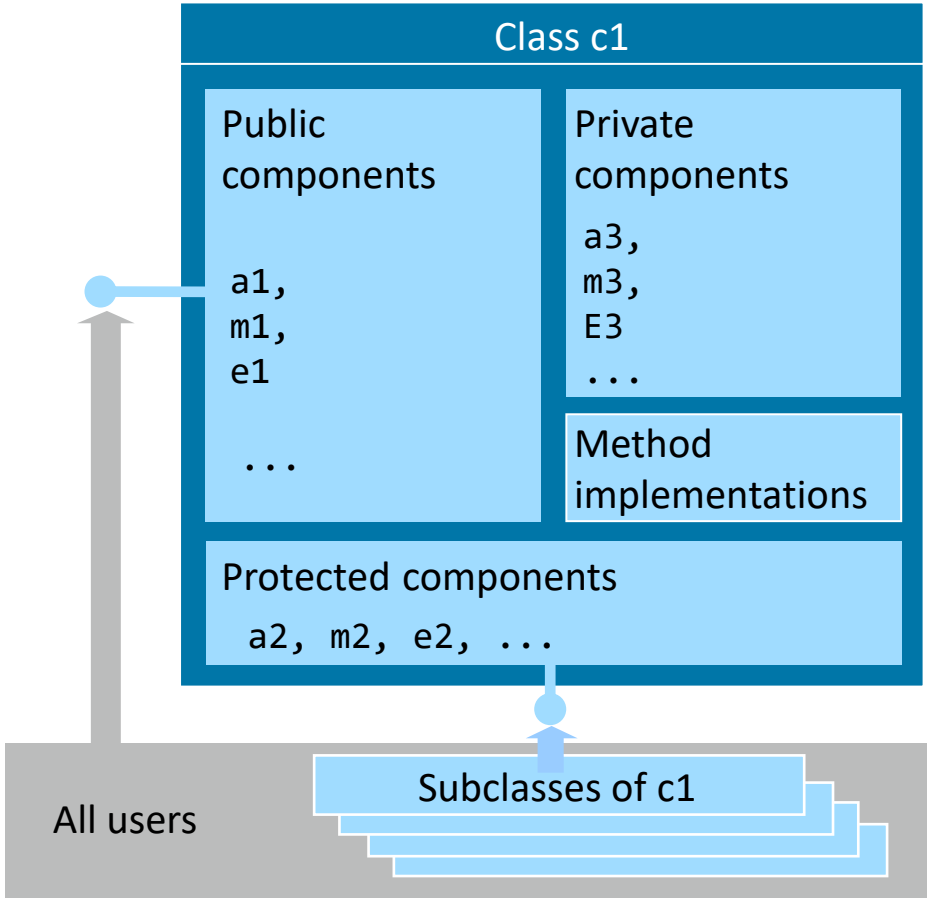


Copyright © 2022 Deloitte Development LLC. All rights reserved.

# Structure of Classes—Visibility Sections

```
CLASS c1 DEFINITION.
 PUBLIC SECTION.
 DATA: a1 …
 METHODS: m1 …
 EVENTS: e1 …
 PROTECTED SECTION.
 DATA: a2 …
 METHODS: m2 …
 EVENTS: e2 …
 PRIVATE SECTION.
 DATA: a3 …
 METHODS: m3 …
 EVENTS: e3 …
ENDCLASS.
```

```
CLASS c1 IMPLEMENTATION.
 METHOD m1. … ENDMETHOD.
 METHOD m2. … ENDMETHOD.
 METHOD m3. … ENDMETHOD.
ENDCLASS.
```






Copyright © 2022 Deloitte Development LLC. All rights reserved.

# Components of Class

A Class basically contains the following:

| | |
|---|---|
| **Attributes** | Any data, constants, and types declared within a class form the attribute of the class. |
| **Methods** | Block of code, providing some functionality offered by the class. Can be compared to function modules. |
| **Events** | A mechanism set within a class that can help a class to trigger methods of other classes. |
| **Interfaces** | Interfaces are independent structures that you can implement in a class to extend the scope of that class. |



Copyright © 2022 Deloitte Development LLC. All rights reserved.

# Components of Classes—Attributes

```
CLASS ... DEFINITION.
  ...
  ... SECTION.
    DATA       ... TYPE ... [READ-ONLY] ...
    CLASS-DATA ... TYPE ... [READ-ONLY] ...
    CONSTANTS  ... TYPE ...  VALUE ...
    ...

ENDCLASS.
```

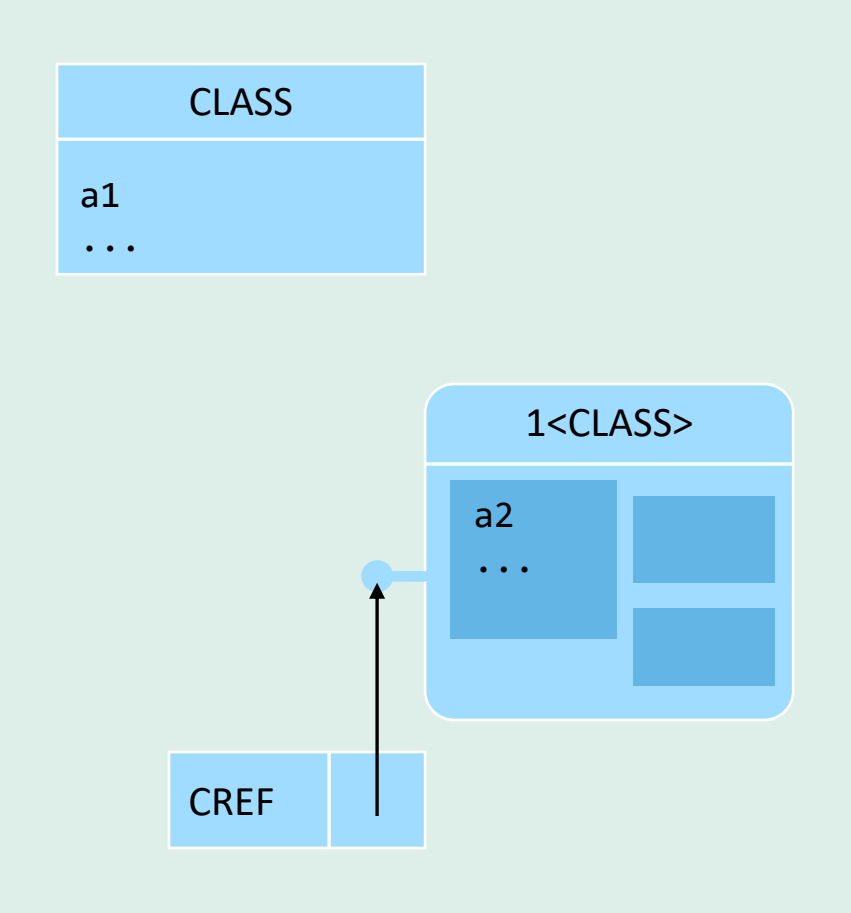
- DATA: Instance attributes
- CLASS-DATA: Static attributes
- CONSTANTS: Constants



Copyright © 2022 Deloitte Development LLC. All rights reserved.

# Static Attributes and Instance Attributes

```
CLASS c DEFINITION.
  PUBLIC SECTION.
    ...
    CLASS-DATA a1(10) TYPE C
               VALUE 'Static'.
    DATA       a2(10) TYPE C
               VALUE 'Instance'.
    ...
ENDCLASS.

DATA: cref TYPE REF TO c.

WRITE c=>a1.
CREATE OBJECT cref TYPE c.
WRITE cref->a2.
```






Copyright © 2022 Deloitte Development LLC. All rights reserved.

# Components of Classes—Methods

- METHODS: Instance methods
- CLASS-METHODS: Static methods

```
CLASS ... DEFINITION.
  ...
  ... SECTION.
    METHODS ... IMPORTING  [VALUE] ... TYPE ...
[OPTIONAL]
                EXPORTING  [VALUE] ... TYPE ...
                CHANGING   [VALUE] ... TYPE ...
[OPTIONAL]
                RETURNING   VALUE(...) TYPE ...
                EXCEPTIONS ...
    CLASS-METHODS ...
    ...
ENDCLASS.
```

```
CLASS ... IMPLEMENTATION.
  METHOD ...
    ...
  ENDMETHOD.
ENDCLASS.
```



Copyright © 2022 Deloitte Development LLC. All rights reserved.

# Constructors

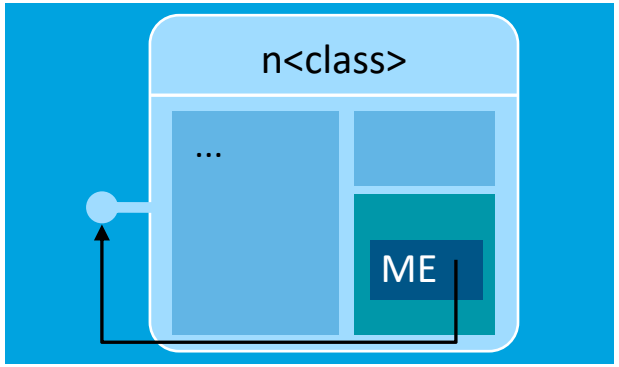
```
CLASS c DEFINITION.
  PUBLIC SECTION.
    METHODS CONSTRUCTOR
            [IMPORTING arg1 TYPE type ... ].

    CLASS-METHODS CLASS_CONSTRUCTOR.

ENDCLASS.
```

```
CLASS c IMPLEMENTATION.
  METHOD CONSTRUCTOR.
    ...
  ENDMETHOD.
  METHOD CLASS_CONSTRUCTOR.
    ...
  ENDMETHOD.
ENDCLASS.
```



Copyright © 2022 Deloitte Development LLC. All rights reserved.

# Accessing the Components of Classes

| | |
|---|---|
| Instance Components | • Instance attribute: ref->attr<br>• Instance method: call method ref->meth |
| Static Components | • Static attribute: class=>attr<br>• Static method: call method class=>meth |
| Special References in Methods | • Self reference: ME->comp<br>• Pseudo reference: SUPER->comp |
| ref->comp | |
| class->comp |  |





Copyright © 2022 Deloitte Development LLC. All rights reserved.

# Local and Global Classes

| Attribute | Global Class | Local Class |
|---|---|---|
| Access | Any program | Only the program where it is defined |
| Storage Location | In class repository | In the program  in which it is defined |
| Creation | Using transaction SE24 | Using SE38 |
| Namespace | Must begin with Y or Z | Can begin with any character |



Copyright © 2022 Deloitte Development LLC. All rights reserved.

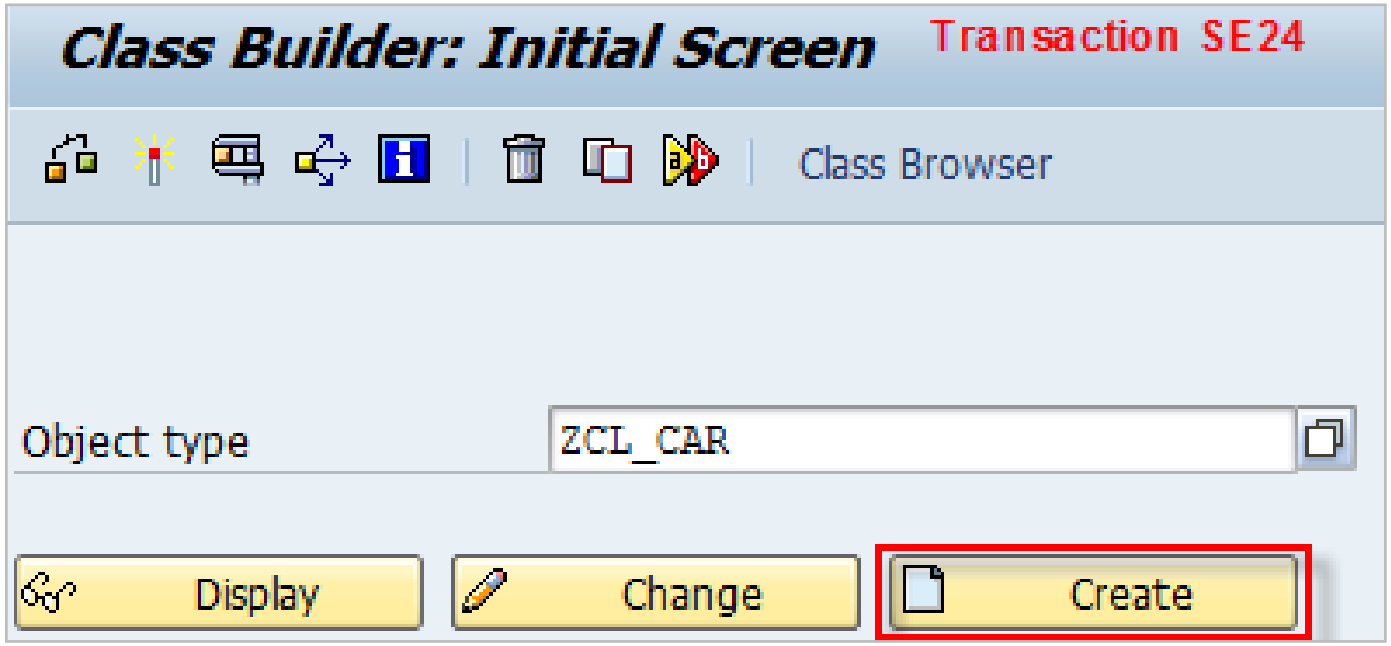
# Global Classes/Interfaces

- Stored in the repository
  - Access from all programs using TYPE REF TO
- Class and interface names governed by the SAP namespace concept
  - Customer namespace: Y* or Z*
- Where-used list available
- Own maintenance tool
  - Transaction SE24—Class Builder

Copyright © 2022 Deloitte Development LLC. All rights reserved.

# Steps for Creating Global Class

- Go to transaction code SE24—Class builder
- Give a class name and click create

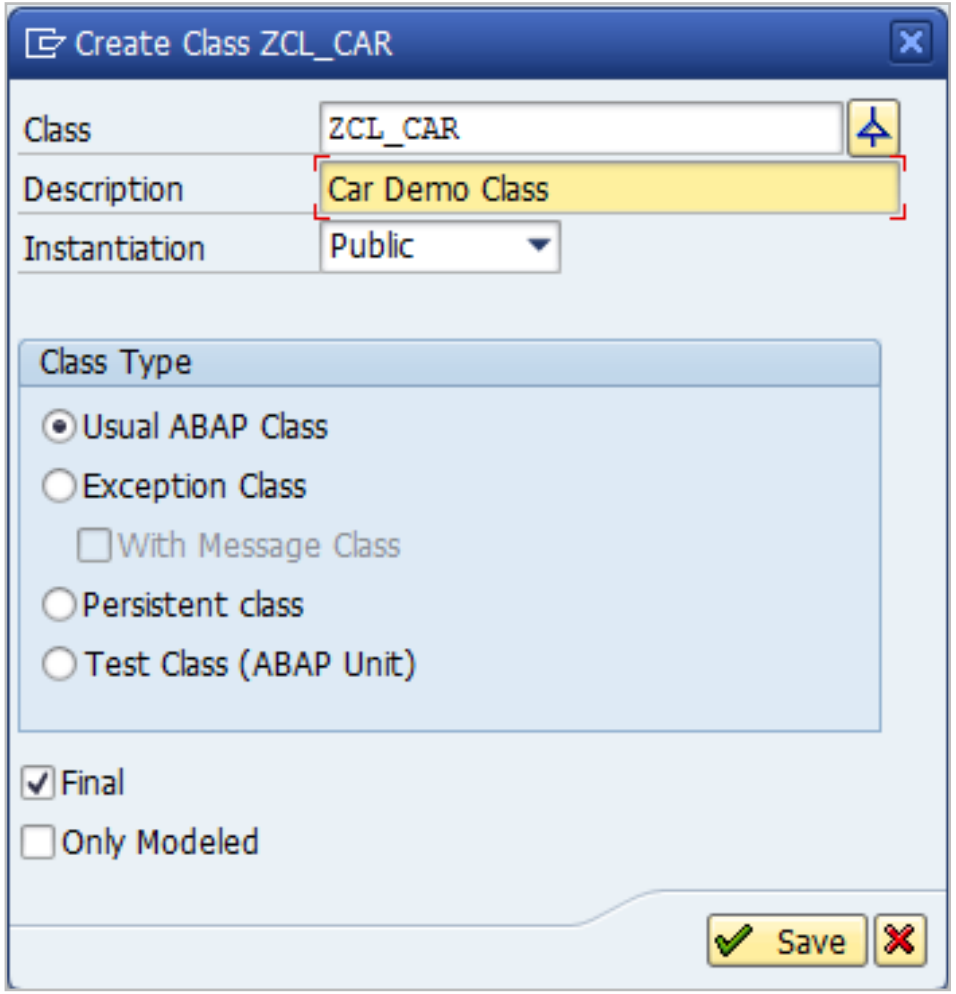





Copyright © 2022 Deloitte Development LLC. All rights reserved.

# Steps for Creating Global Class (Cont.)

- Give a description of the class
- Select instantiation as Public
- Select the radio button as Usual ABAP class
- Check the checkbox Final

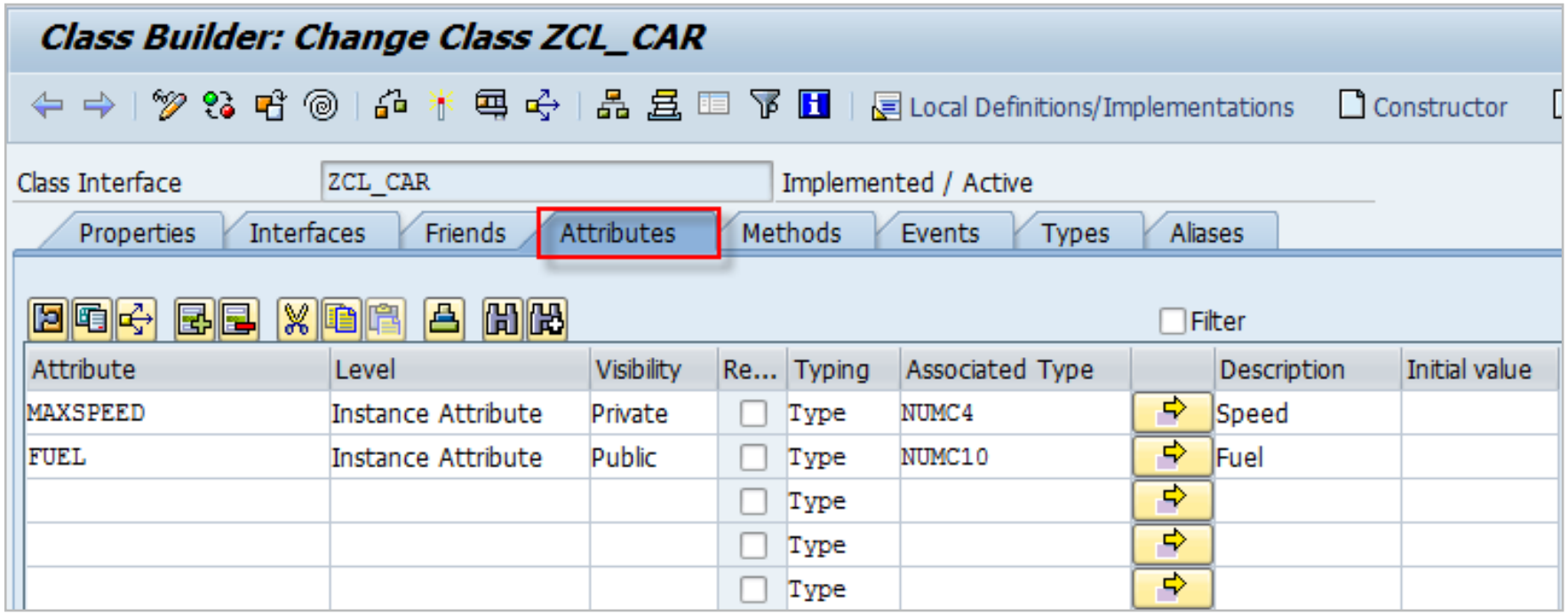





Copyright © 2022 Deloitte Development LLC. All rights reserved.

# Steps for Creating Global Class (Cont.)

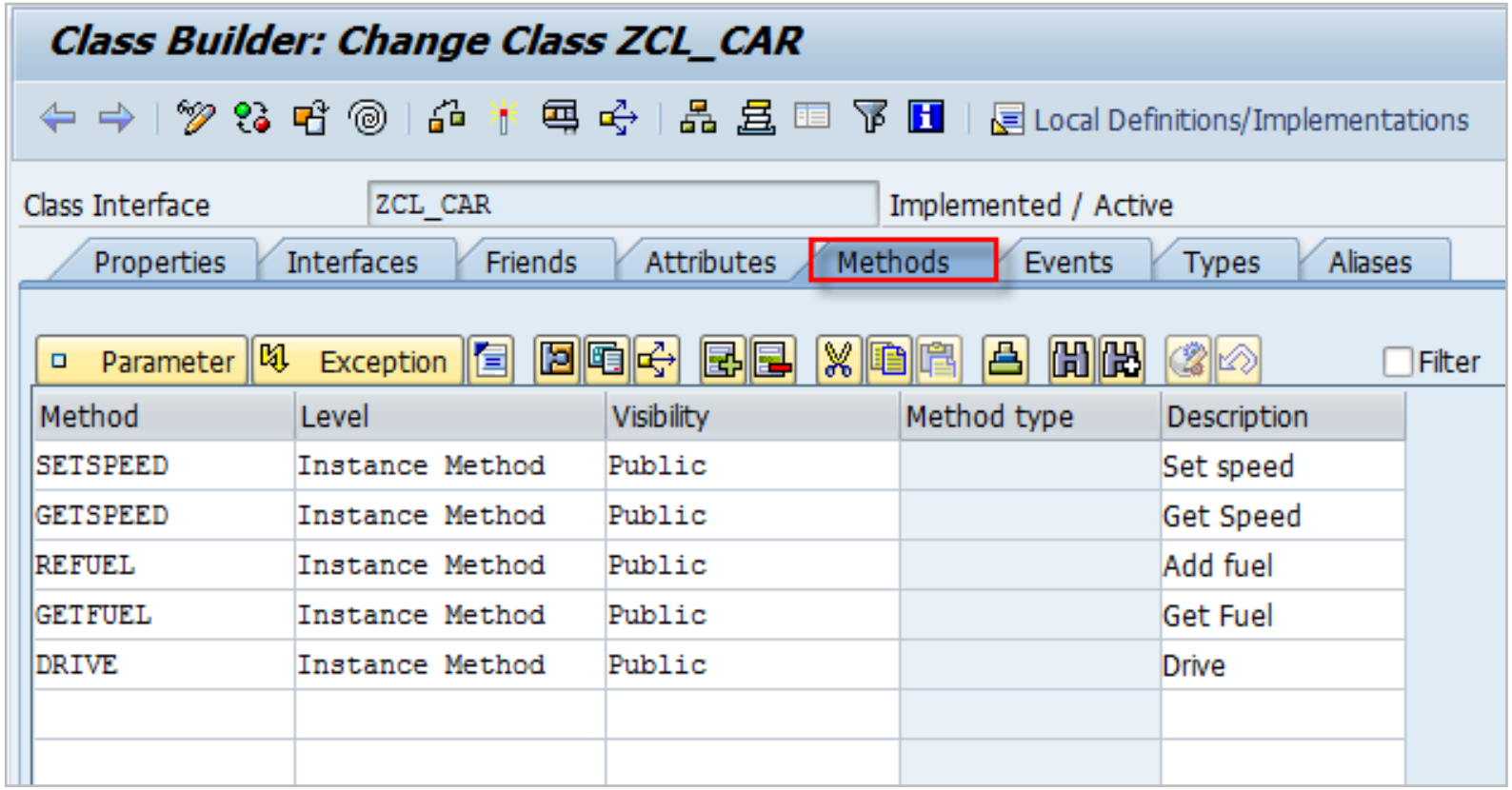
Class Attributes




Copyright © 2022 Deloitte Development LLC. All rights reserved.

# Steps for Creating Global Class (Cont.)

Class Methods

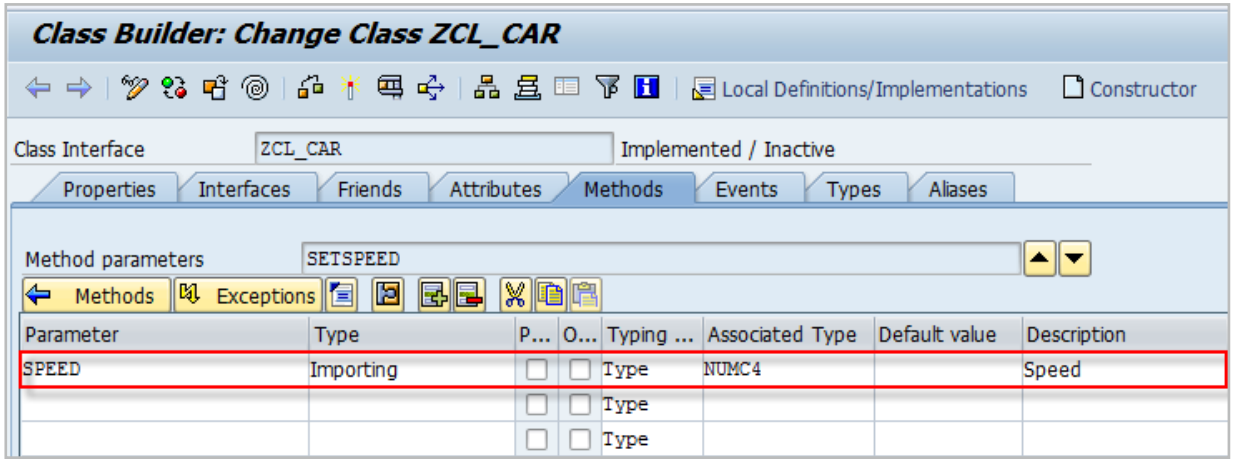


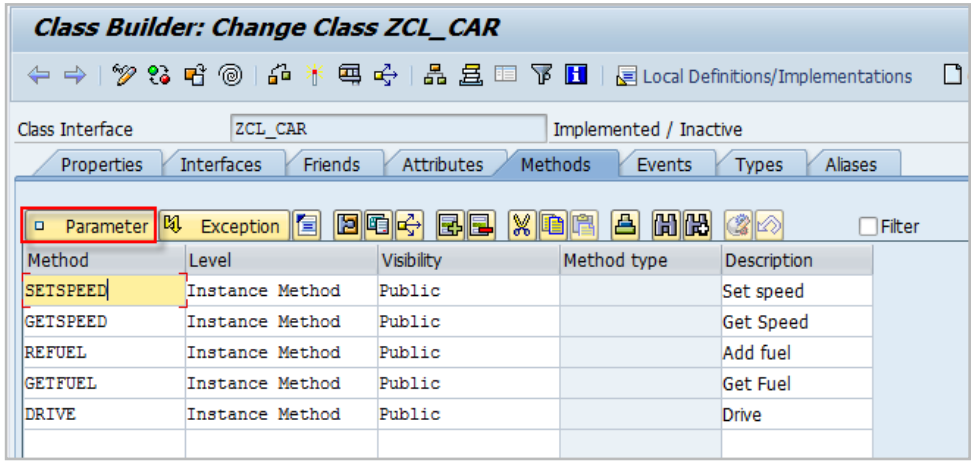

| Method | Level | Visibility | Method type | Description |
|---|---|---|---|---|
| SETSPEED | Instance Method | Public | | Set speed |
| GETSPEED | Instance Method | Public | | Get Speed |
| REFUEL | Instance Method | Public | | Add fuel |
| GETFUEL | Instance Method | Public | | Get Fuel |
| DRIVE | Instance Method | Public | | Drive |

Copyright © 2022 Deloitte Development LLC. All rights reserved.

# Steps for Creating Global Class (Cont.)

- Method interface
- Select a method and choose parameters



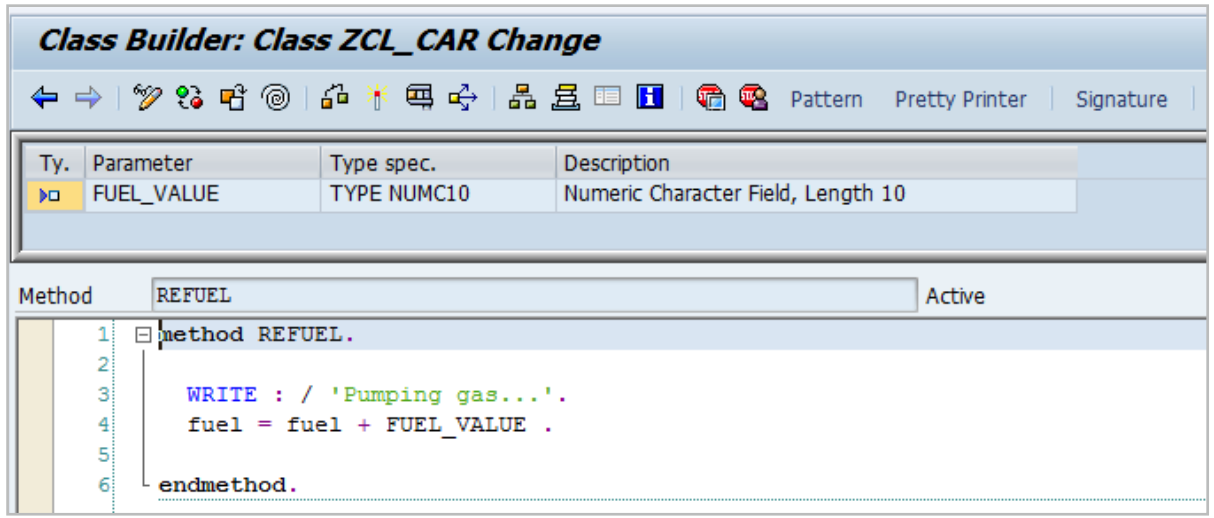



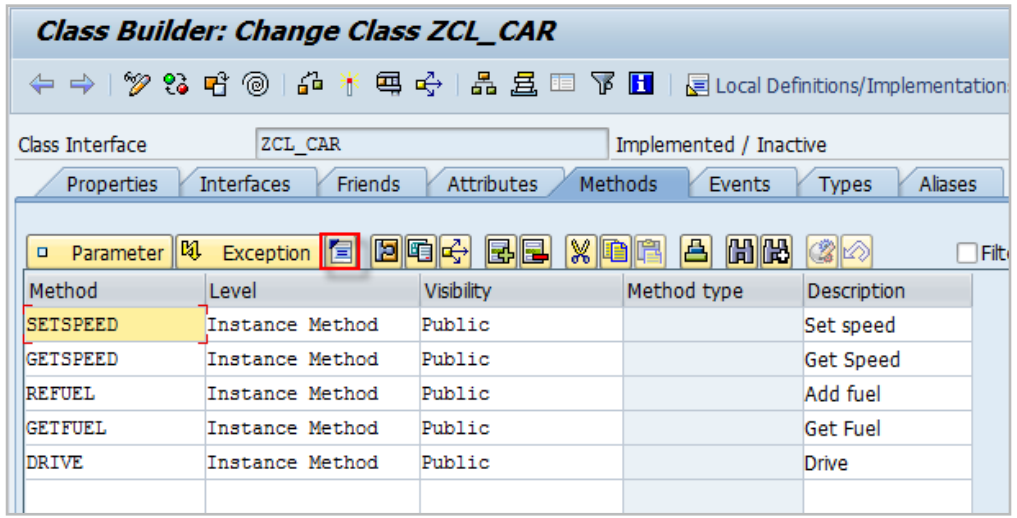



Copyright © 2022 Deloitte Development LLC. All rights reserved.

# Steps for Creating Global Class (Cont.)

Method implementation



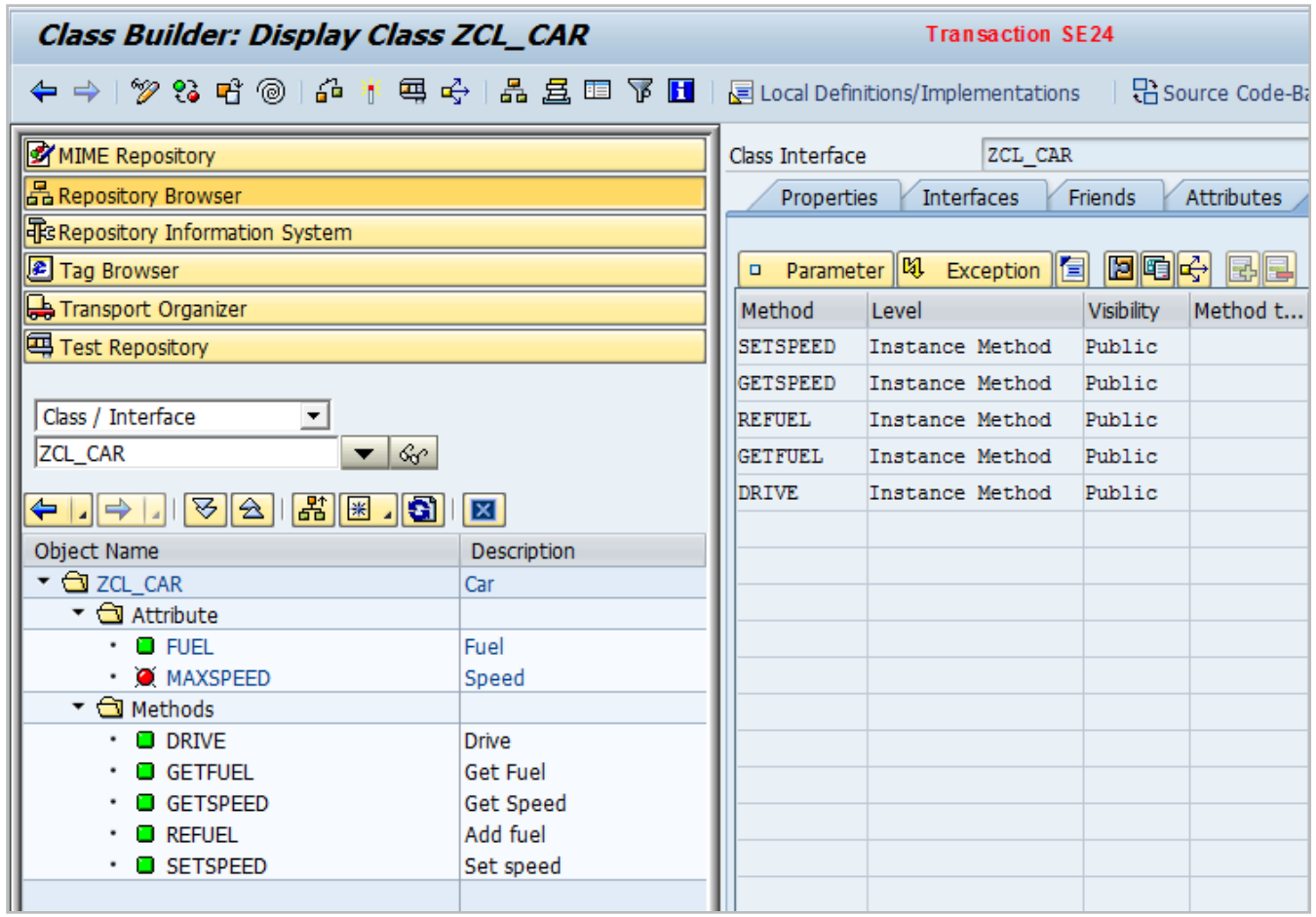






Copyright © 2022 Deloitte Development LLC. All rights reserved.

# Working With Global Classes

Display the global class in the object navigator






Copyright © 2022 Deloitte Development LLC. All rights reserved.

# Working With Global Classes (Cont.)

Instantiate the class in program

```
*&---------------------------------------------------------------------*
*& Report  ZCL_CAR_DEMO
*&
*&---------------------------------------------------------------------*
REPORT  zcl_car_demo.


DATA : gcl_polo   TYPE REF TO zcl_car,
       gcl_escort TYPE REF TO zcl_car.

DATA : gv_fuel TYPE numc10.


START-OF-SELECTION.

*=The code below creates an object called polo and escort,
* both of class type car.
  CREATE OBJECT gcl_polo.

  CREATE OBJECT gcl_escort.
```

Copyright © 2022 Deloitte Development LLC. All rights reserved.

# Working With Global Classes (Cont.)

Calling global class method

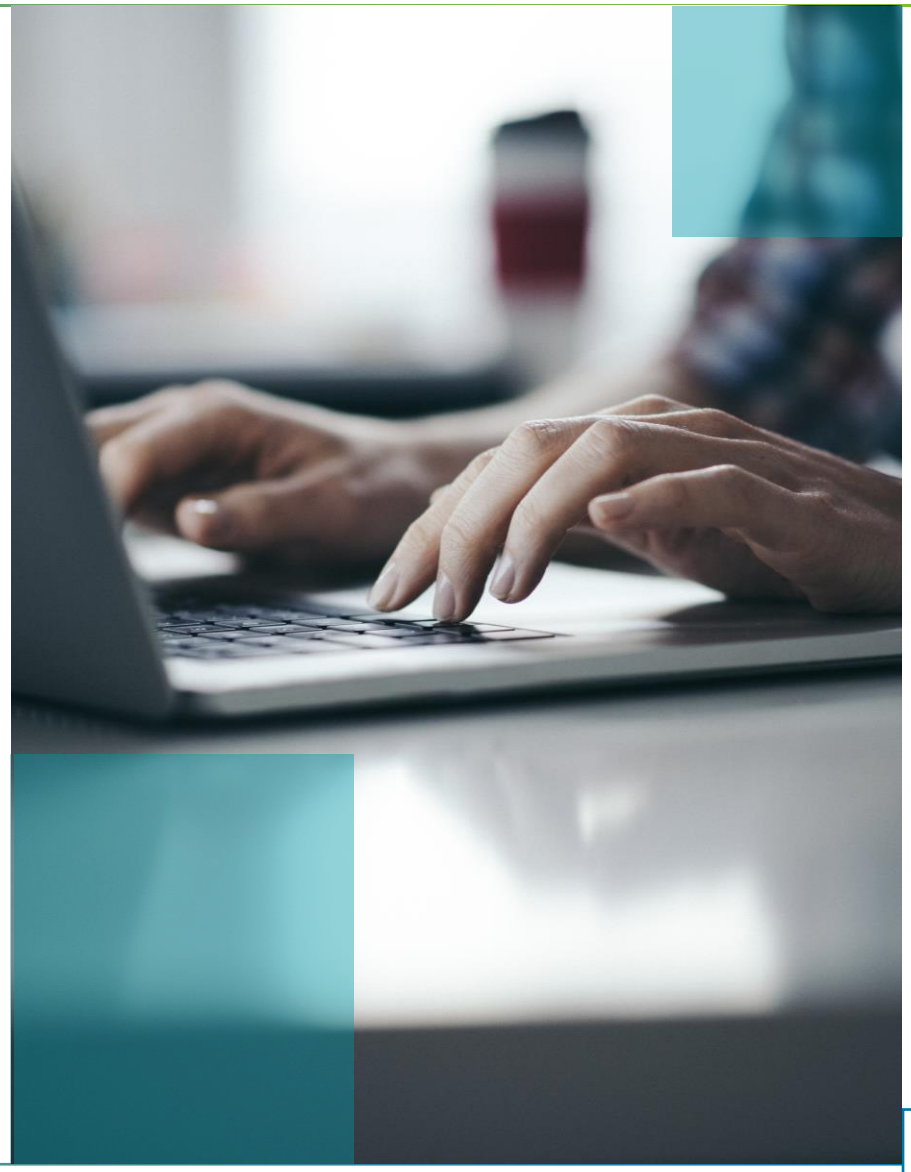
```
*=Refuel - Polo
  CALL METHOD gcl_polo->refuel
    EXPORTING
      fuel_value = 100.

*=Refuel - Escort
  CALL METHOD gcl_escort->refuel
    EXPORTING
      fuel_value = 50.


*=Drive - Polo & Escort
  DO 10 TIMES.
    CALL METHOD gcl_polo->drive .

    CALL METHOD gcl_escort->drive.
  ENDDO.

*=Fuel Polo status
  CALL METHOD gcl_polo->getfuel
    IMPORTING
      fuel_value = gv_fuel.
  WRITE : / 'Polo Fuel Staus', gv_fuel.

*=Fuel Escort status
  CLEAR : gv_fuel.
  CALL METHOD gcl_escort->getfuel
    IMPORTING
      fuel_value = gv_fuel.
  WRITE : / 'Escort Fuel Staus', gv_fuel.
```



Copyright © 2022 Deloitte Development LLC. All rights reserved.

# When to Use Local Classes?

- When the user wants to enhance the class methods i.e., Pre, Post or Overwrite Exits, the system would create a local class to achieve the functionality.
- While working on some standalone programs to receive local events. E.g., DOUBLE_CLICK from CL_GUI_ALV_GRID.
- While using the ALV grid on screen using the OO approach.

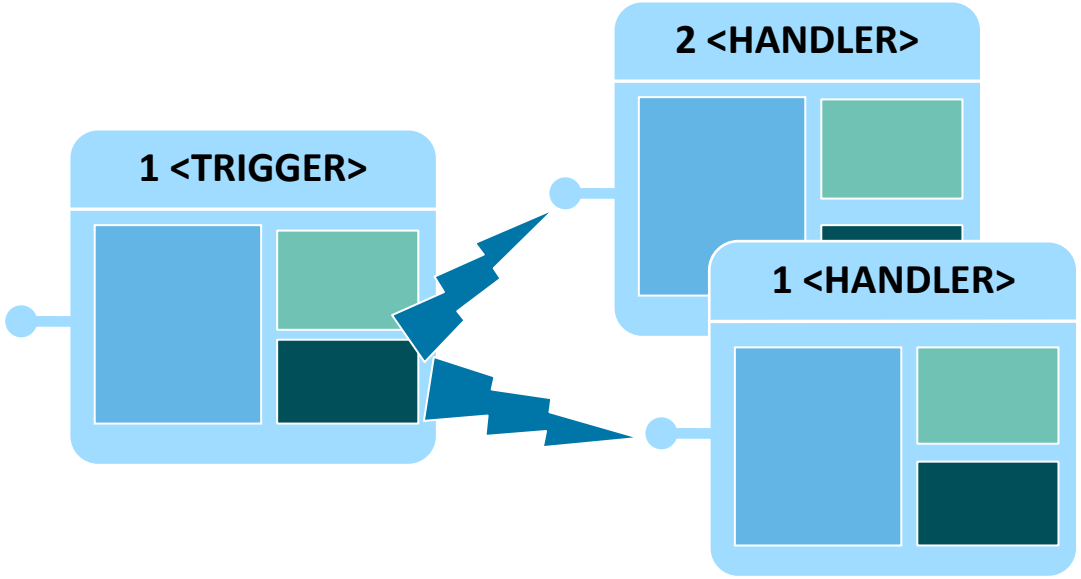




Copyright © 2022 Deloitte Development LLC. All rights reserved.

# What are Events?

- Events are components of classes.
- Methods can raise the events of their class.
- Handler methods can be triggered by events.






Copyright © 2022 Deloitte Development LLC. All rights reserved.

# Events Overview

</>

**Event Trigger**

```
CLASS c1 DEFINITION.
 PUBLIC SECTION.
 EVENTS e1 EXPORTING VALUE(p1)
 TYPE i.
 METHODS m1.
 PRIVATE SECTION.
 DATA a1 TYPE i.
ENDCLASS.
CLASS c1 IMPLEMENTATION.
 METHOD m1.
 a1 = ...
 RAISE EVENT e1
 EXPORTING p1 = a1.
 ENDMETHOD.
ENDCLASS.
```

**Event Handler**

```
CLASS c2 DEFINITION.
 PUBLIC SECTION.
 METHODS m2 FOR EVENT e1 OF c1
 IMPORTING p1.
 PRIVATE SECTION.
 DATA a2 TYPE i.
ENDCLASS.
CLASS C2 IMPLEMENTATION.
 METHOD m2.
 a2 = p1.
 ...
 ENDMETHOD.
ENDCLASS.
```



Copyright © 2022 Deloitte Development LLC. All rights reserved.

# Declaring and Triggering Events

```
CLASS ... DEFINITION.

  ... SECTION.

    METHODS ...

    EVENTS ... [EXPORTING VALUE(...) TYPE ... [OPTIONAL]].

    CLASS-EVENTS ...

ENDCLASS.
```

```
CLASS ... IMPLEMENTATION.

  METHOD ...
    ...
    RAISE EVENT ... EXPORTING ... = ...
    ...
  ENDMETHOD.

ENDCLASS.
```



Copyright © 2022 Deloitte Development LLC. All rights reserved.

# Event Handler Methods

```
CLASS ... DEFINITION.

  ... SECTION.

    METHODS ... FOR EVENT ... OF ... [IMPORTING
... SENDER ... ].

ENDCLASS.

PROGRAM ...

DATA: trigger   TYPE REF TO trigger,
      handler_1 TYPE REF TO handler,
      handler_2 TYPE REF TO handler.
CREATE OBJECT: trigger, handler_1, handler_2.
SET HANDLER handler_1->handle_event
            handler_2->handle_event FOR trigger.

CALL METHOD trigger->raise_event.
```

```
CLASS ... IMPLEMENTATION.

  METHOD ...
    ...
  ENDMETHOD.

ENDCLASS.
```



Copyright © 2022 Deloitte Development LLC. All rights reserved.

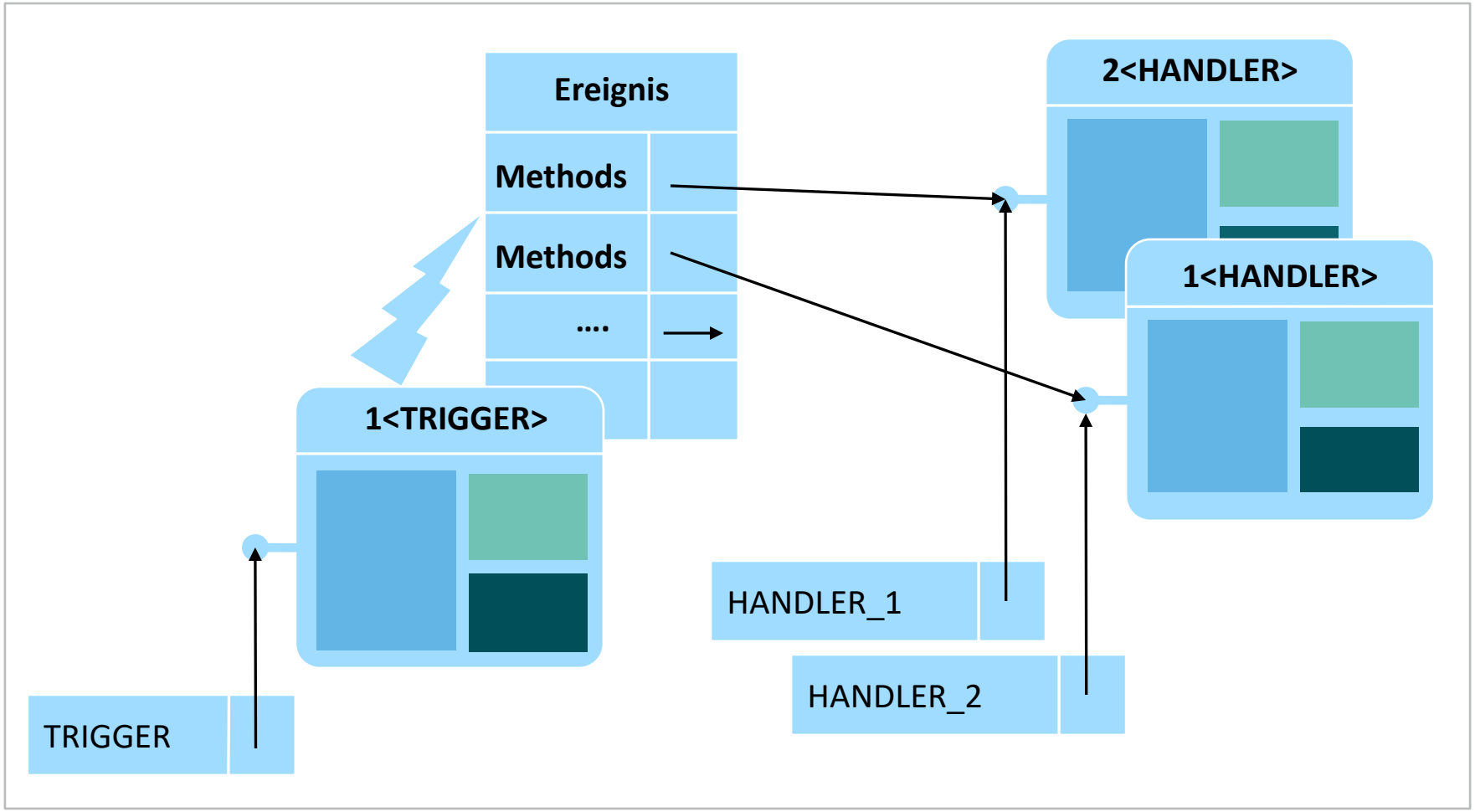
# Event Handling—Registration

```
PROGRAM ...

DATA: trigger   TYPE REF TO trigger,
      handler_1 TYPE REF TO handler,
      handler_2 TYPE REF TO handler.

CREATE OBJECT: trigger, handler_1, handler_2.

SET HANDLER handler_1->handle_event
            handler_2->handle_event FOR trigger.

CALL METHOD trigger->raise_event.
```



Copyright © 2022 Deloitte Development LLC. All rights reserved.

# Handling Events—References



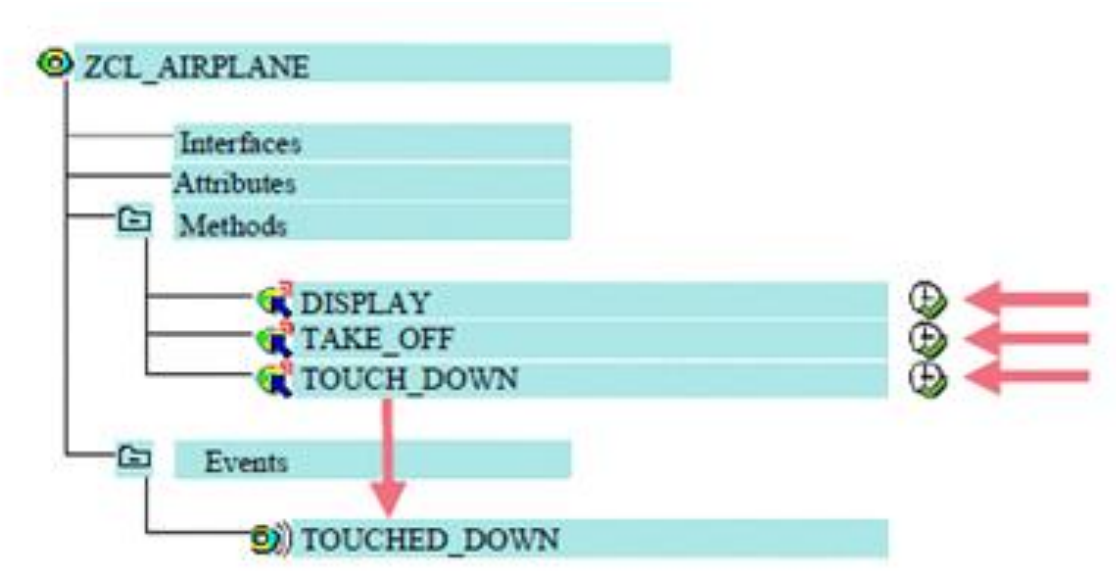



Copyright © 2022 Deloitte Development LLC. All rights reserved.

# Class Builder—Testing Classes

- Test Methods
- Trigger Events

**TEST ENVIRONMENT**

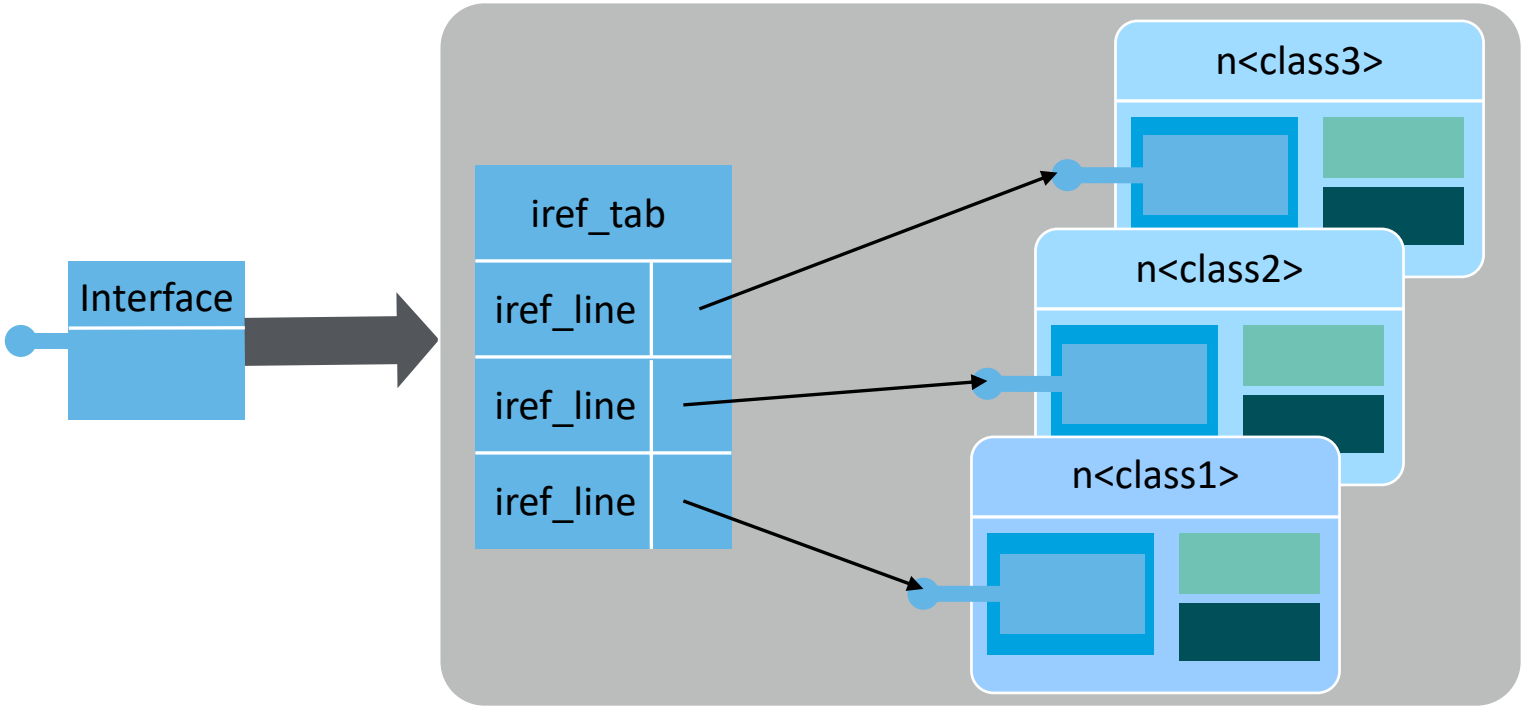





Copyright © 2022 Deloitte Development LLC. All rights reserved.

# Interfaces

- Definition of an interface without implementation
- Classes can implement several interfaces
- Uniform access with interface references
- Polymorphism independent from inheritance





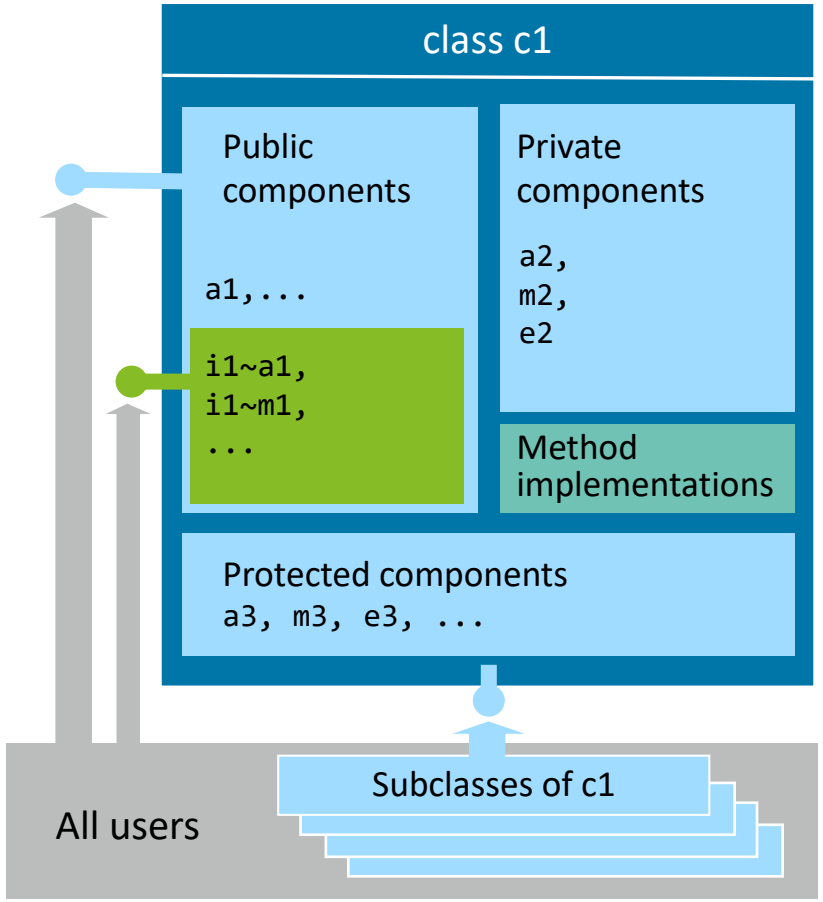

Copyright © 2022 Deloitte Development LLC. All rights reserved.

# Interfaces Overview

```
INTERFACE i1.
 DATA:    a1 ...
 METHODS: m1 ...
 EVENTS:  e1 ...
ENDINTERFACE.
```

```
CLASS c1 DEFINITION.
 PUBLIC SECTION.
 DATA a1 ...
 INTERFACES i1 ...
 PROTECTED SECTION.
 PRIVATE SECTION.
ENDCLASS.
```

```
CLASS c1 IMPLEMENTATION.
 METHOD i1~m1. … ENDMETHOD.
ENDCLASS.
```






Copyright © 2022 Deloitte Development LLC. All rights reserved.

# Interfaces Definition

```
INTERFACE ... .
 ...
 DATA: ... TYPE ... [READ-ONLY] ...
 CLASS-DATA: ... TYPE ... [READ-ONLY] ...
 CONSTANTS: ... TYPE ... [VALUE ...]

 METHODS: ... IMPORTING [VALUE] ... TYPE ... [OPTIONAL]
 EXPORTING [VALUE] ... TYPE ...
 CHANGING [VALUE] ... TYPE ... [OPTIONAL]
 RETURNING VALUE(...) TYPE ...
 EXCEPTIONS ...
 CLASS-METHODS: ...

 EVENTS: ... [EXPORTING VALUE(...) TYPE ... [OPTIONAL]].
 CLASS-EVENTS:...

 INTERFACES: ...

ENDINTERFACE.
```



Copyright © 2022 Deloitte Development LLC. All rights reserved.

# Interfaces Implementation

```
CLASS ... IMPLEMENTATION.

  METHOD ...~...
    ...
  ENDMETHOD.

ENDCLASS.
```

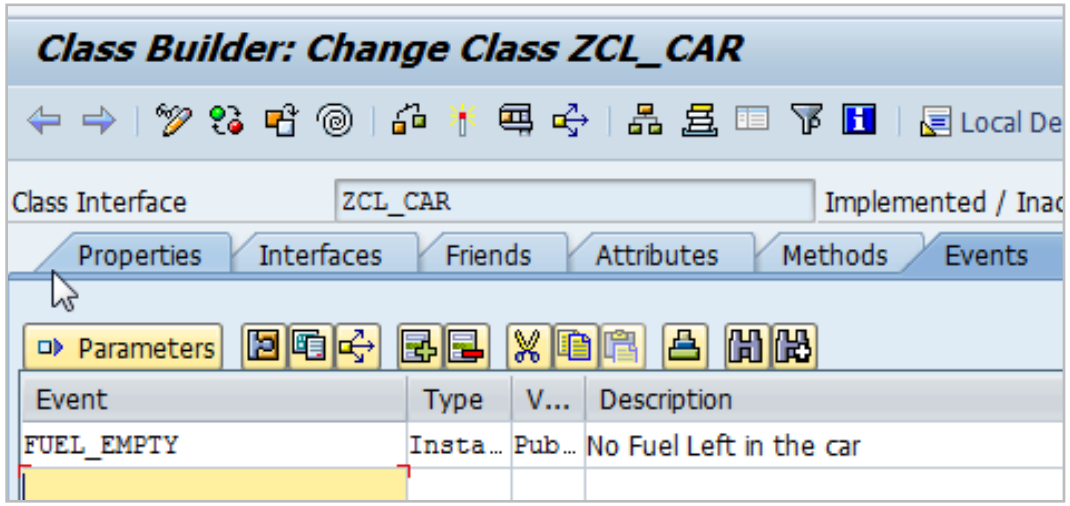
```
CLASS ... DEFINITION.

  PUBLIC SECTION.
    INTERFACES: ...
    ...
  ...

ENDCLASS.
```

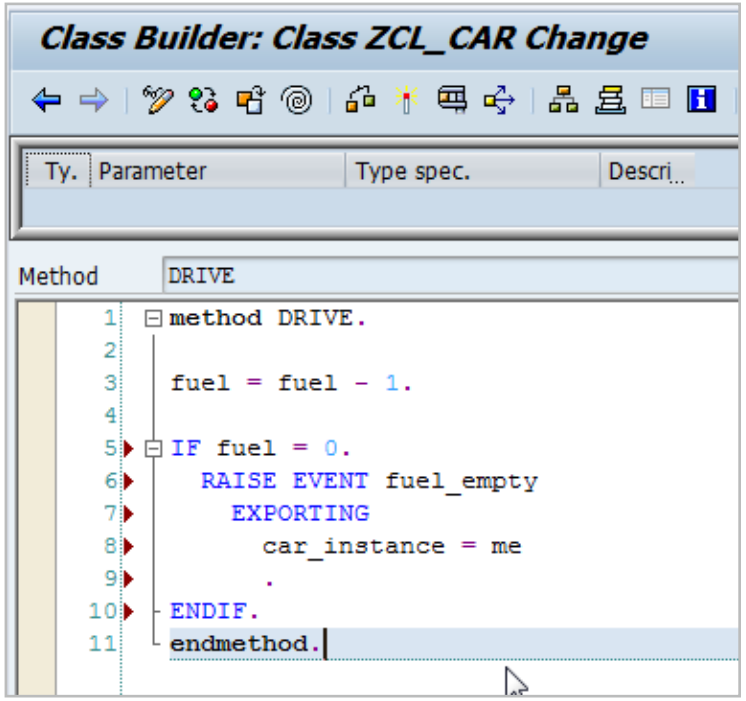


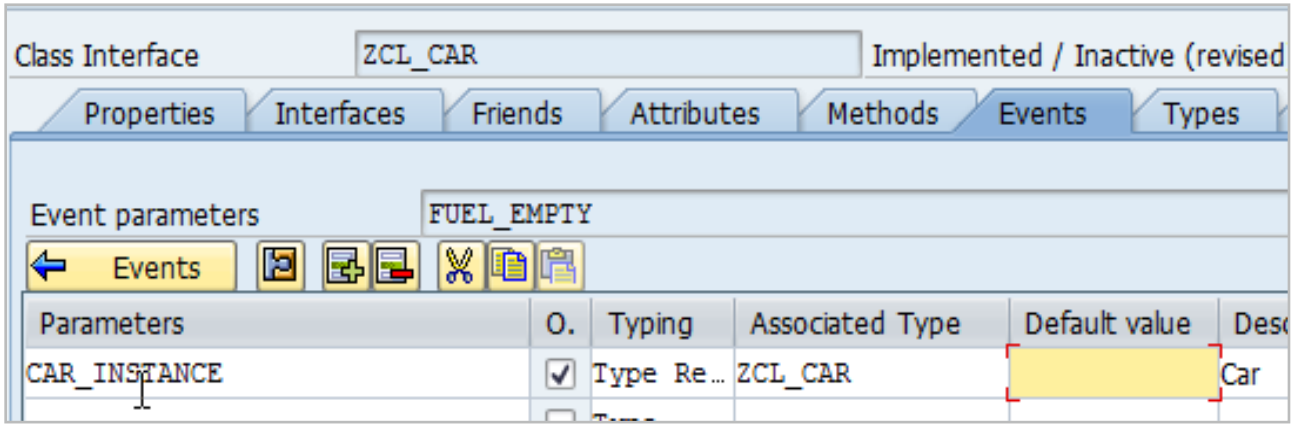
Copyright © 2022 Deloitte Development LLC. All rights reserved.

# Demo

- Add an Event to the class
- Add a parameter to the event
- Modify the method Drive to trigger the event






Copyright © 2022 Deloitte Development LLC. All rights reserved.

# Demo (Cont.)

- Create a local class to act as a fuel station
- Define an event handling method to handle the event
- Implement the method to refuel the car when the fuel is empty
- Drive away in the car and watch as it refuels

```
*&---------------------------------------------------------------------*
REPORT ztrain00_fuel_station.
*---------------------------------------------------------------------*
*       CLASS FUEL_STATION DEFINITION
*---------------------------------------------------------------------*
*
*---------------------------------------------------------------------*
CLASS fuel_station DEFINITION.
  PUBLIC SECTION.
    METHODS: reload_fuel FOR EVENT fuel_empty OF zcl_car IMPORTING car_instance.
ENDCLASS.                    "fuel_station DEFINITION
```

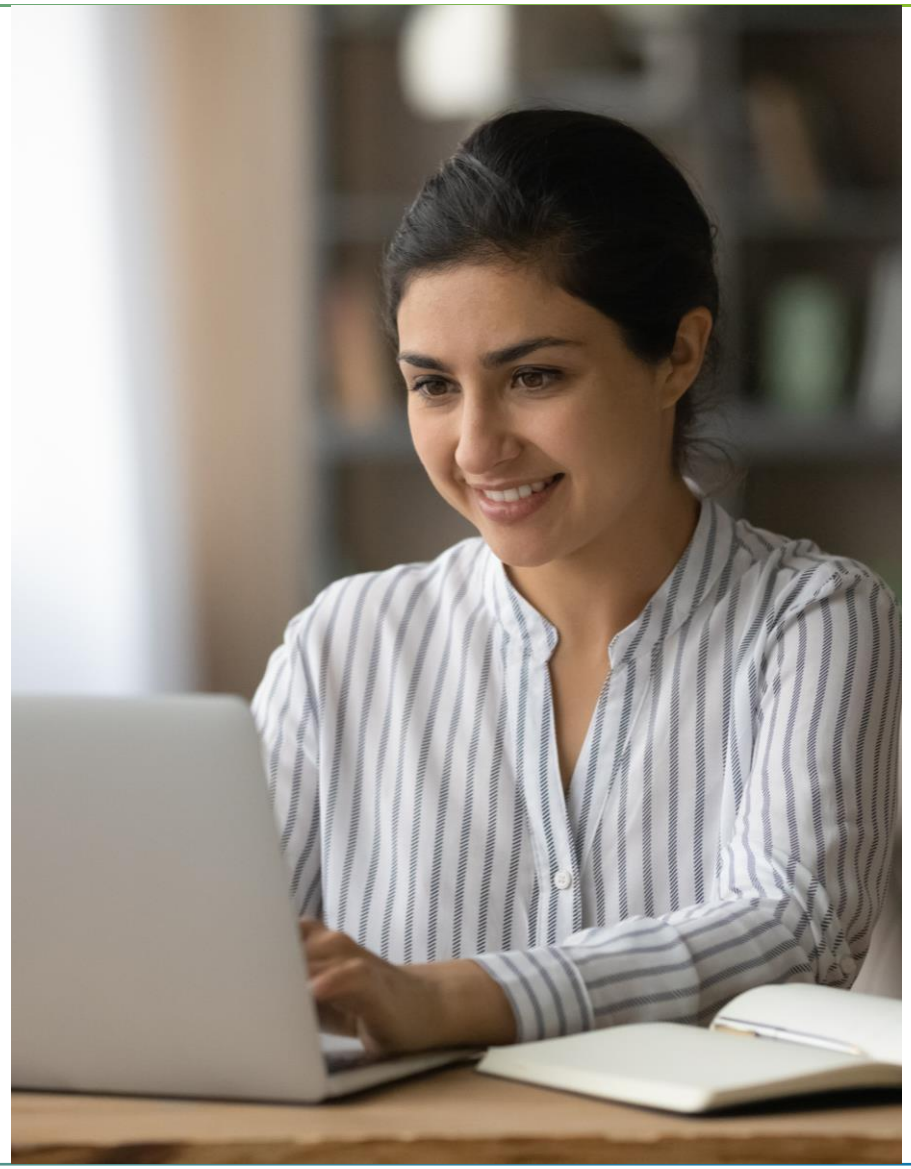
```
CLASS fuel_station IMPLEMENTATION.

  METHOD reload_fuel.

    car_instance->refuel( fuel_value = 10 ).
  ENDMETHOD.                    "RELOAD_FUEL
```

```
START-OF-SELECTION.
CREATE OBJECT: lr_fuel_station, lr_car.
SET HANDLER lr_fuel_station->reload_fuel FOR lr_car.
lr_car->refuel( fuel_value = 10 ).
DO 30 TIMES.
 lr_car->getfuel( IMPORTING fuel_value = lv_fuel  ).
 WRITE: /,lv_fuel.
lr_car->drive( ).
ENDDO.
```



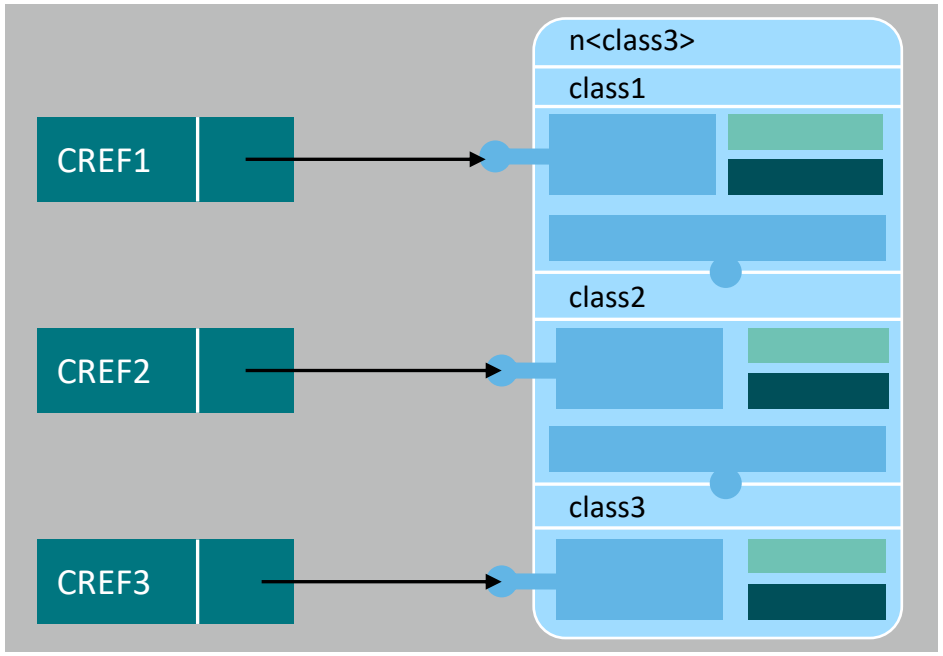
Copyright © 2022 Deloitte Development LLC. All rights reserved.

# Inheritance



# Introduction

- Definition of a class by inheriting the components from a superclass (Reuse)
- Specialization by adding own components and redefining methods in subclasses
- Polymorphism by accessing subclass objects




Copyright © 2022 Deloitte Development LLC. All rights reserved.

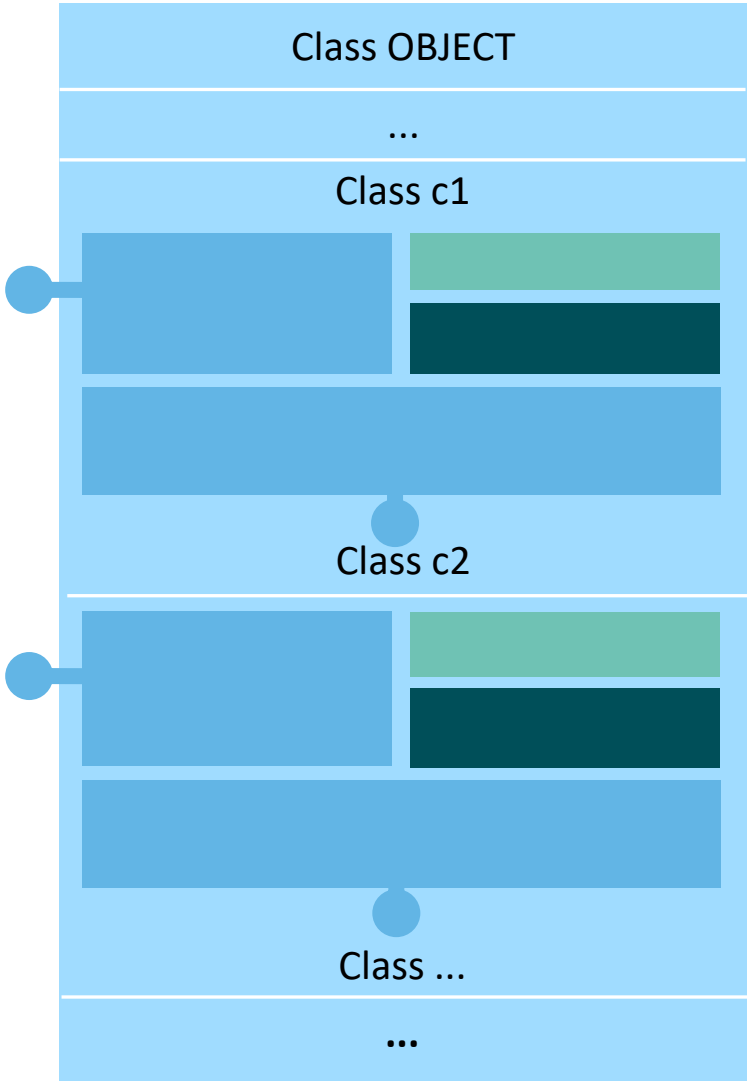
# Overview

```
CLASS c1 DEFINITION INHERITING FROM ...
 ...
ENDCLASS.
```

```
CLASS c1 IMPLEMENTATION.
 ...
ENDCLASS.
```

```
CLASS c2 DEFINITION INHERITING FROM c1.
 ...
ENDCLASS.
```

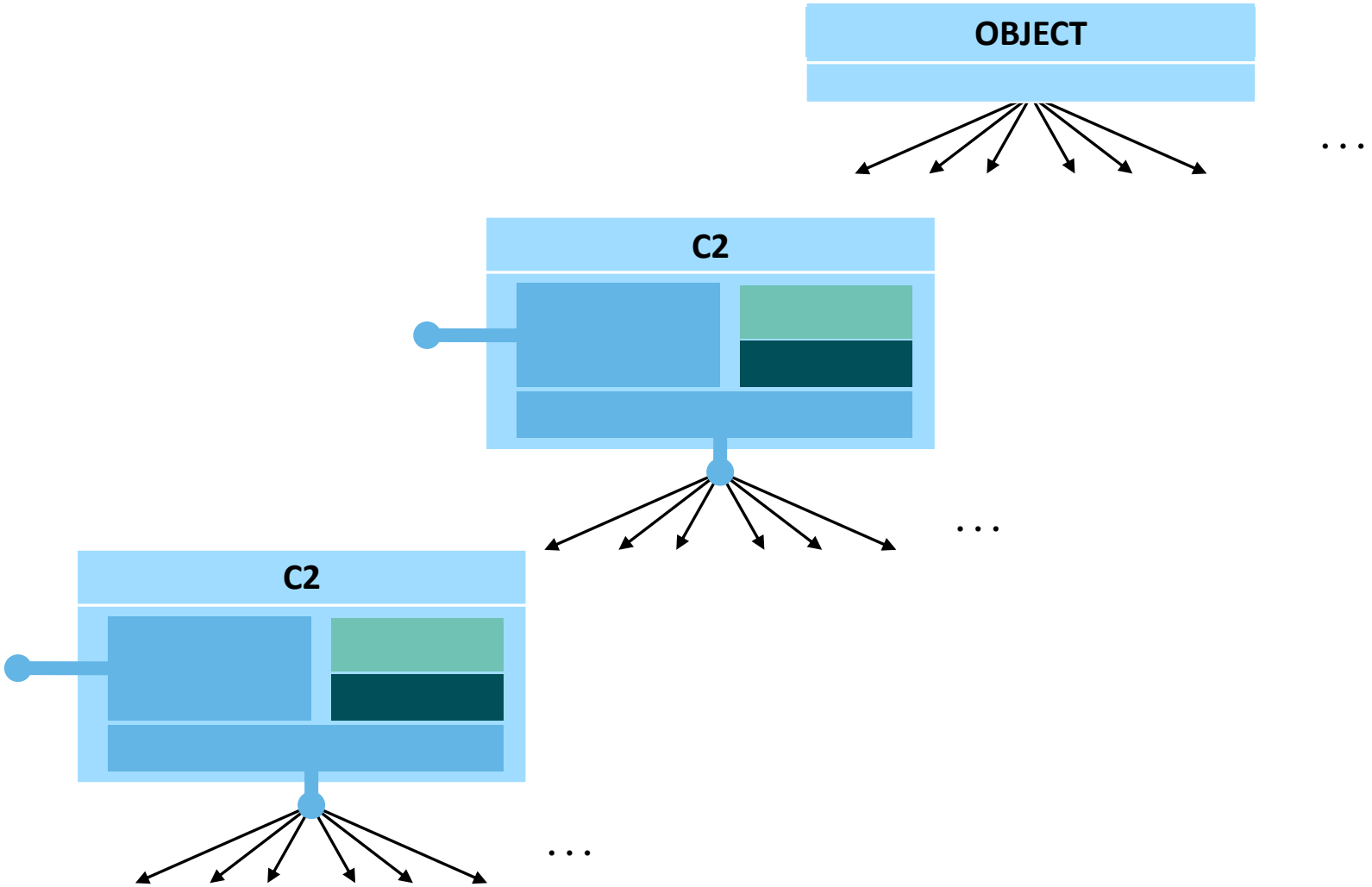
```
CLASS c2 IMPLEMENTATION.
 ...
ENDCLASS.
```






Copyright © 2022 Deloitte Development LLC. All rights reserved.

# Single Inheritance






Copyright © 2022 Deloitte Development LLC. All rights reserved.

# Redefining Methods

```
CLASS ... DEFINITION INHERITING FROM ...

 ... SECTION.
 METHODS ... REDEFINITON ...
 ...
 ...

ENDCLASS.
```

```
CLASS ... IMPLEMENTATION.

 METHOD ...
 ...
 ENDMETHOD.

ENDCLASS.
```

**Semantic rules**

- Subclasses must behave just like their superclass for all users of inherited components
- A redefined method must observe the original semantics
- Inheritance should only be used to specialize



Copyright © 2022 Deloitte Development LLC. All rights reserved.

# Inheritance and Constructors

```
CLASS subclass DEFINITION INHERITING FROM superclass.
 PUBLIC SECTION.
 METHODS CONSTRUCTOR IMPORTING ...
 ...
ENDCLASS.
```

**Access to static attributes only**

**Access to instance attributes also**

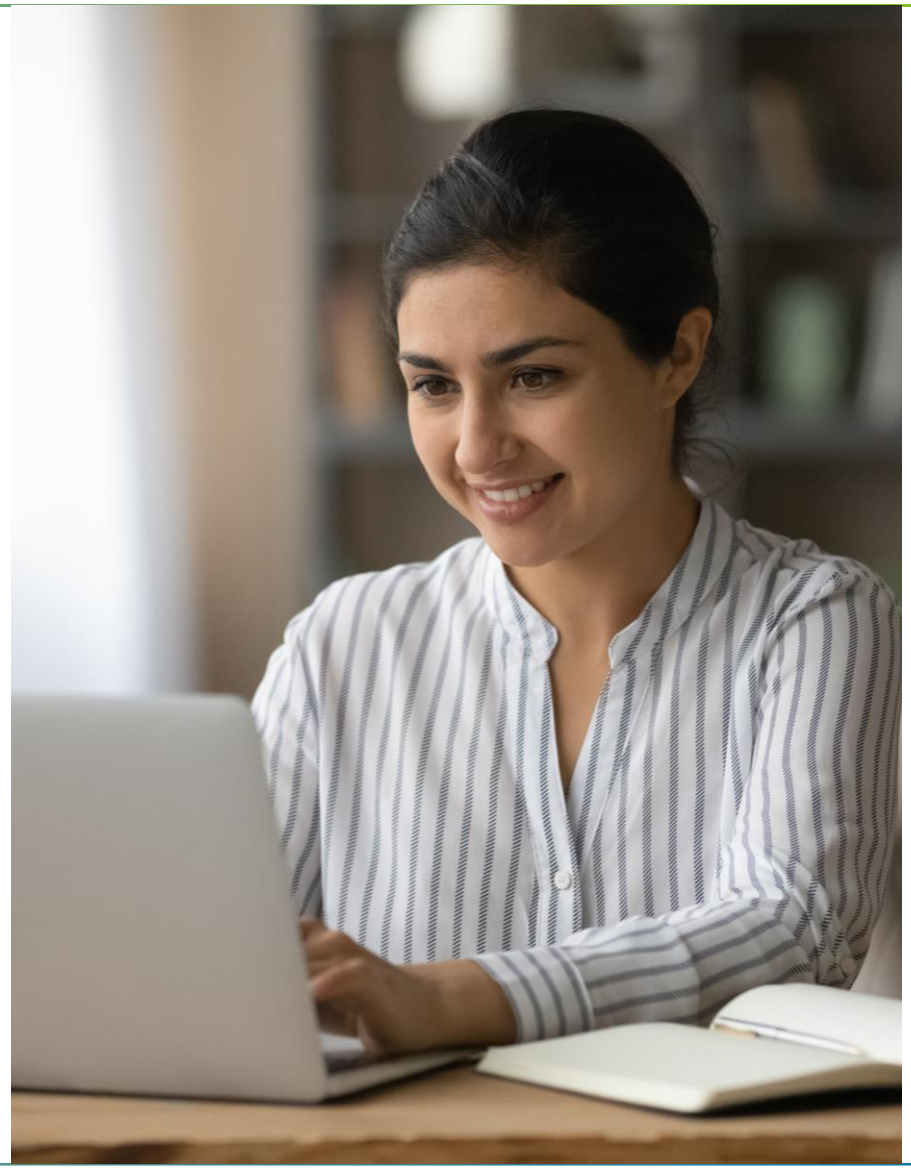
```
CLASS subclass IMPLEMENTATION.
  METHOD CONSTRUCTOR.
    ...
    CALL METHOD SUPER->CONSTRUCTOR EXPORTING ...
    ...
  ENDMETHOD.
  ...
ENDCLASS.
```

```
PROGRAM ...

DATA o1 TYPE REF TO subclass.
CREATE OBJECT o1 TYPE subclass EXPORTING ...
```



Copyright © 2022 Deloitte Development LLC. All rights reserved.

# Class Exceptions



# Exception Handling

What happens if a class-based exception occurs?

The system interrupts the normal program flow and tries to find a suitable handler. If it does not find a handler, a runtime error occurs.

- Handlers are defined for one or more exception classes. To define handlers for all the subordinate classes of the exception classes, a superclass is specified. This enables entire groups of related exceptions to be handled.



Copyright © 2022 Deloitte Development LLC. All rights reserved.

# Syntax

```
TRY.

…

CATCH cx_... cx_... cx_... [INTO ref_exc1].

…

CATCH cx_... cx_... cx_... [INTO ref_exc2].

…

CLEANUP.

…

ENDTRY.
```

An exception can only be handled if causing statements(s) is enclosed in a TRY-ENDTRY block.

Handlers for the specified exception classes and their subclasses.

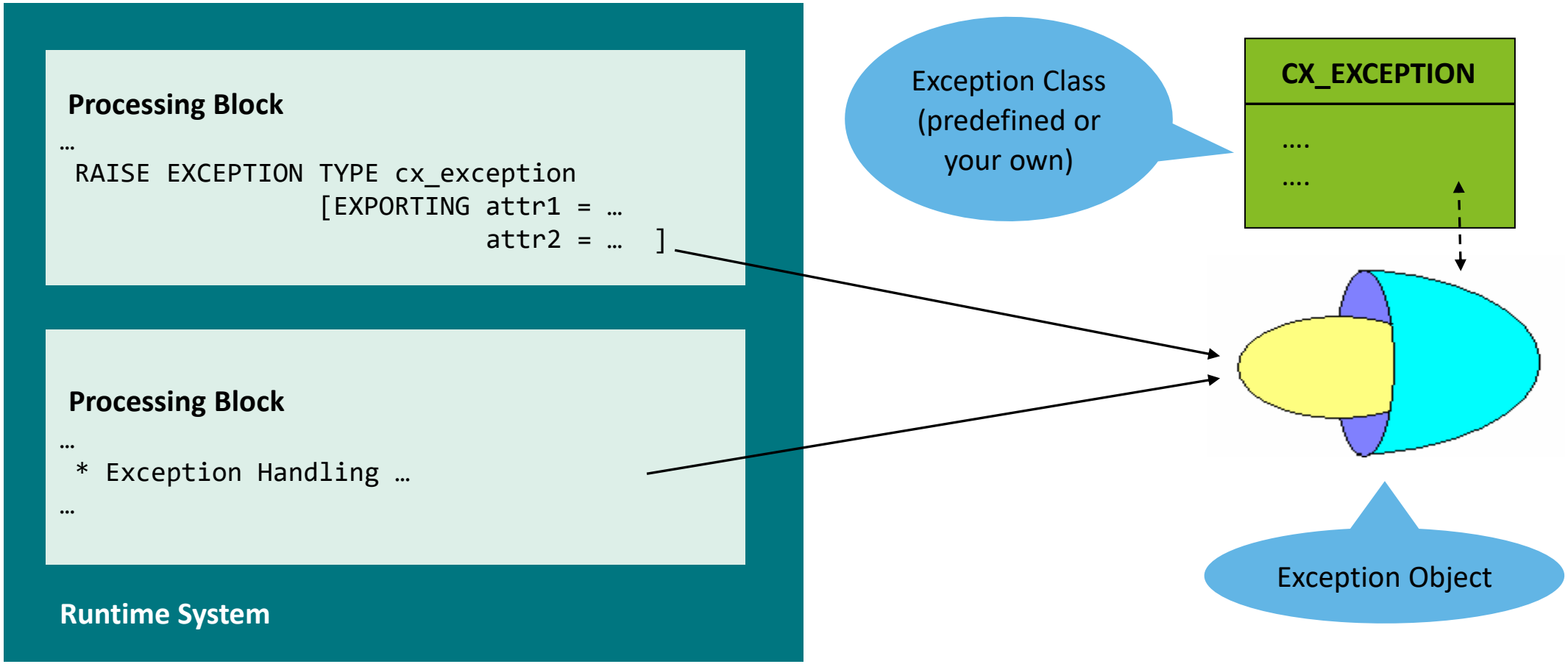
Statements to be executed before exception is propagated, if there is no exception handler.



Copyright © 2022 Deloitte Development LLC. All rights reserved.

# Example

When a class-based exception is raised, the system interrupts normal program flow and tries to navigate to the suitable handler. If a handler is not found, a runtime error occurs.






Copyright © 2022 Deloitte Development LLC. All rights reserved.

# Keywords

</>

| TRY Block | Catch Block |
| --- | --- |
| The TRY block contains the application coding whose exceptions are to be handled. | A catch block contains the exception handler that is executed when a particular exception has occurred in the TRY block of the same TRY-ENDTRY structure.<br>• The syntax for introducing an exception handler is:<br>– CATCH cx_... cx_... INTO ref.<br>– With the INTO addition, a reference to the exception object can be placed as a reference variable. This enables the attributes of the exception object to be accessed in the handler.<br>– What should be the type of this ref variable? |

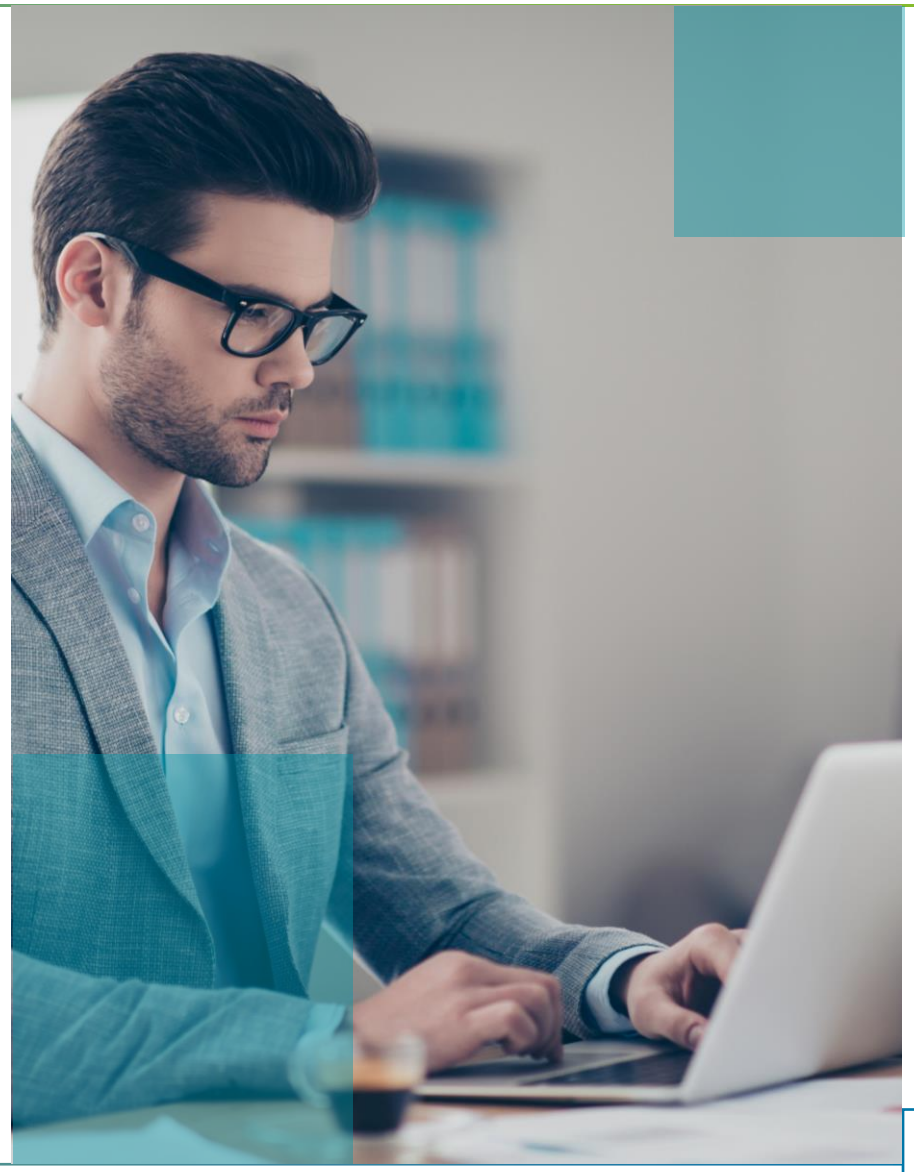
Its static type must be the exception class itself or one of its super classes



Copyright © 2022 Deloitte Development LLC. All rights reserved.

# TRY-CATCH Block

- TRY-ENDTRY structures can be nested.
- If an exception occurs in a TRY block, the system first searches for a CATCH statement (which will handle the exception) in the same block and then step by step upwards in all enclosing TRY-ENDTRY blocks. If it finds one, it navigates to this handler, else if the TRY-ENDTRY block is in a procedure, it then tries to propagate the exception to the calling program.
- After the exception occurs, the system searches through the listed exception handlers in the specified order and executes the first handler whose CATCH statement contains the relevant exception class or one of its super classes.

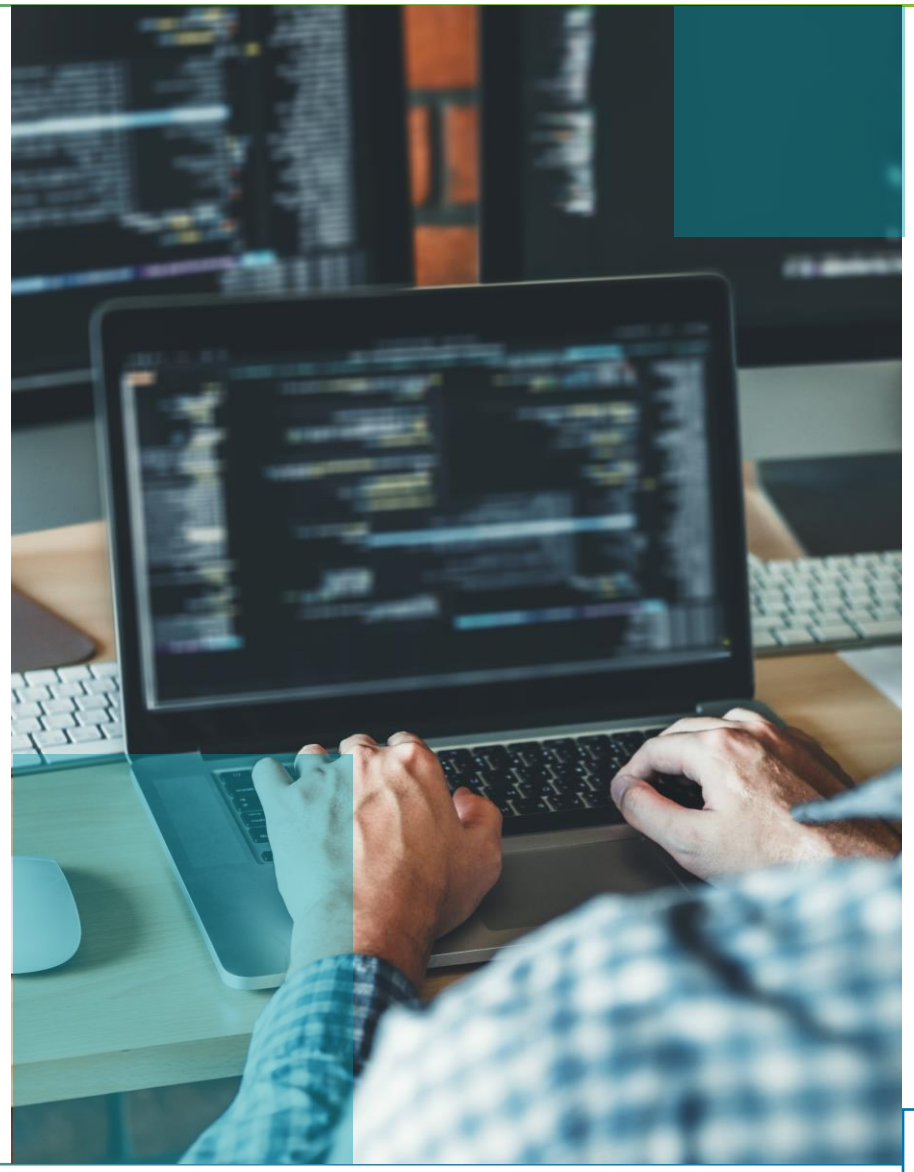




Copyright © 2022 Deloitte Development LLC. All rights reserved.

# Exception Classes

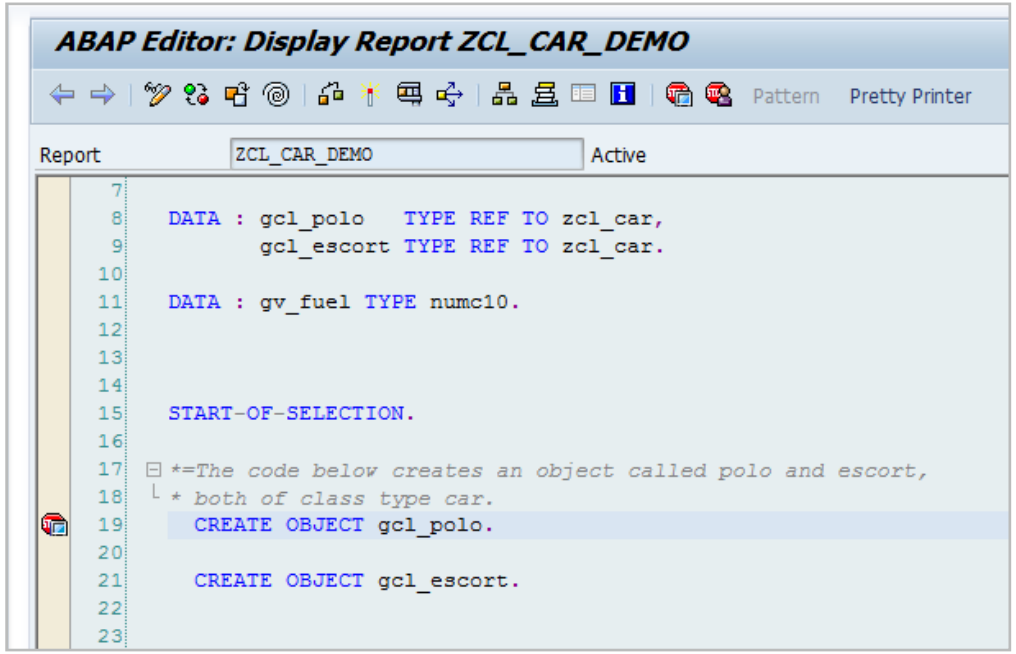
- The CX_ROOT class provides some predefined methods that are inherited by all exception classes:
  - GET_SOURCE_POSITION: The method returns the name of the program and line number in the source code where the exception occurred.
  - GET_TEXT: The method returns an exception text to be displayed in the form of a string.
- Exception Classes inheriting from these super classes have new components, usually individual exception texts.
- Exceptions occurring in procedures need not be handled where they occur but can be propagated to the caller of the procedure, by using RAISING addition in the procedure's signature.

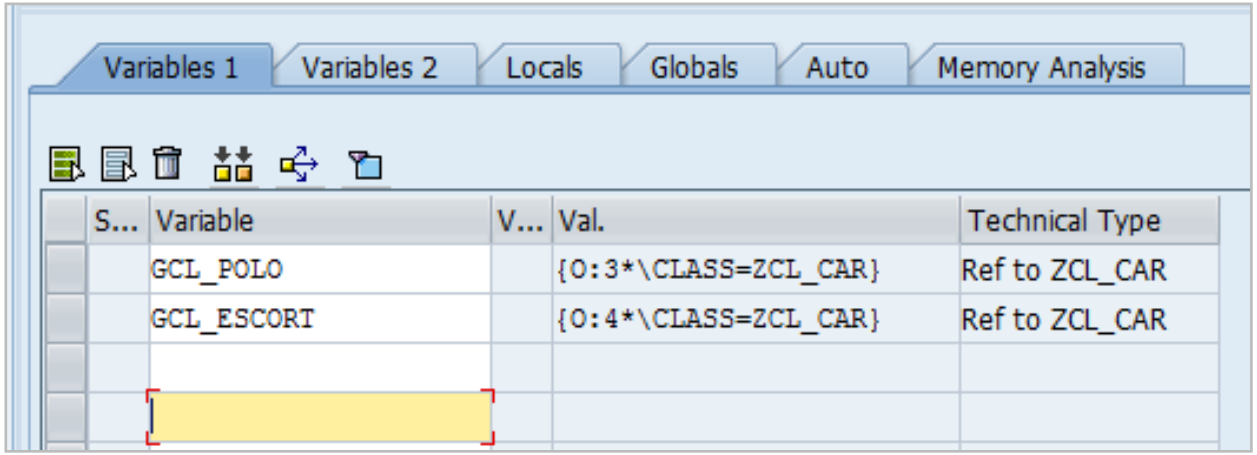




Copyright © 2022 Deloitte Development LLC. All rights reserved.

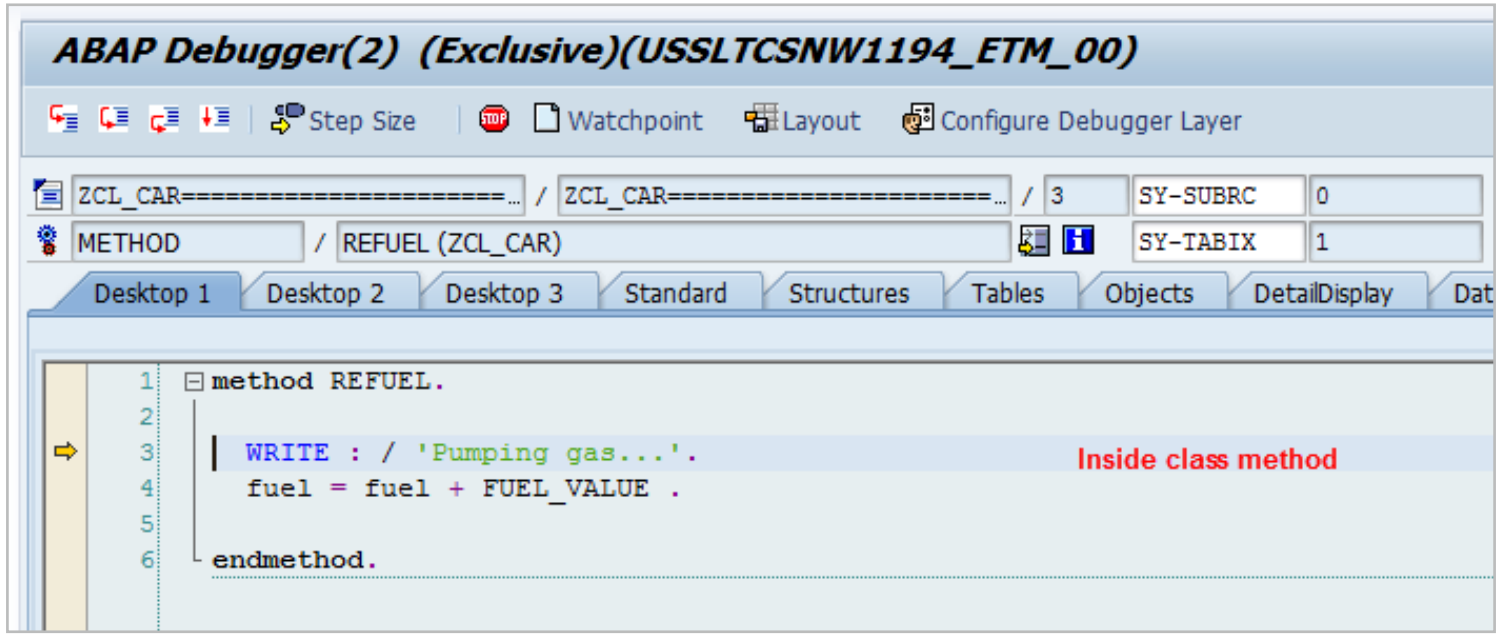
# Debugging: Session Breakpoints in Constructor and Methods

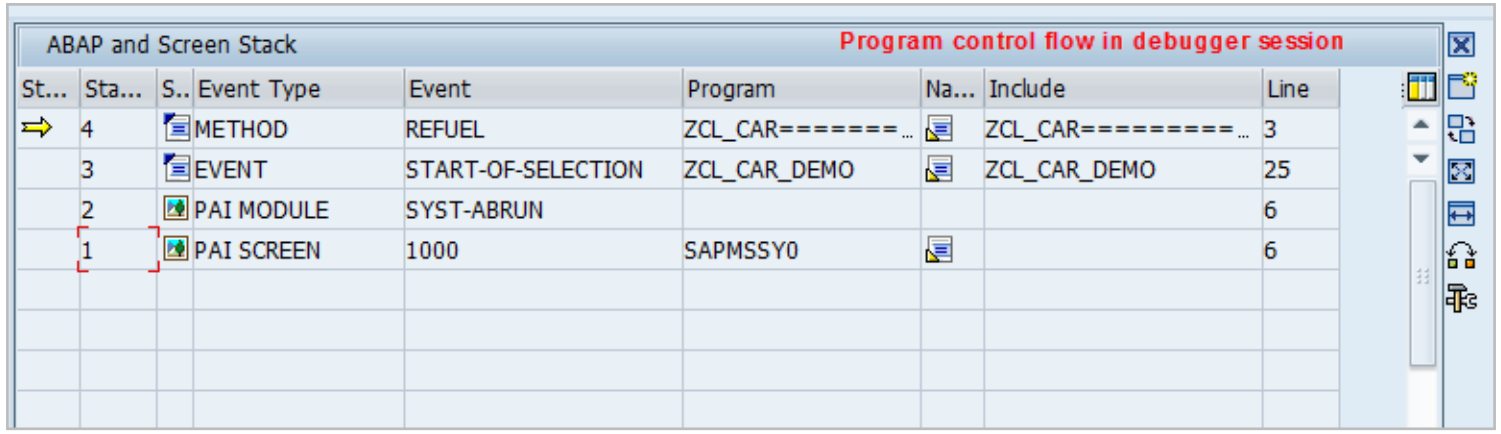



Copyright © 2022 Deloitte Development LLC. All rights reserved.

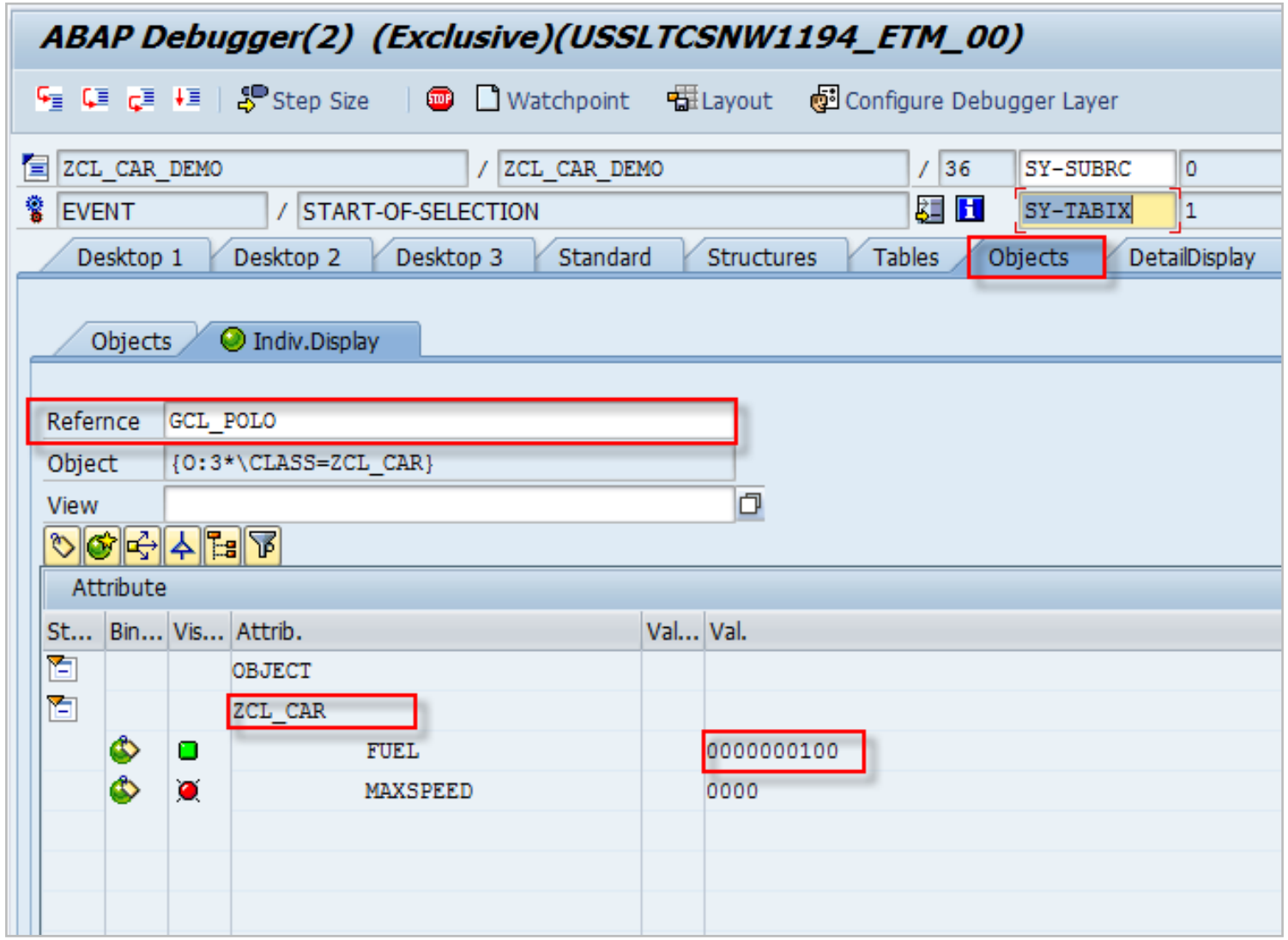
# Debugging: Session Breakpoints in Constructor and Methods (Cont.)




Copyright © 2022 Deloitte Development LLC. All rights reserved.

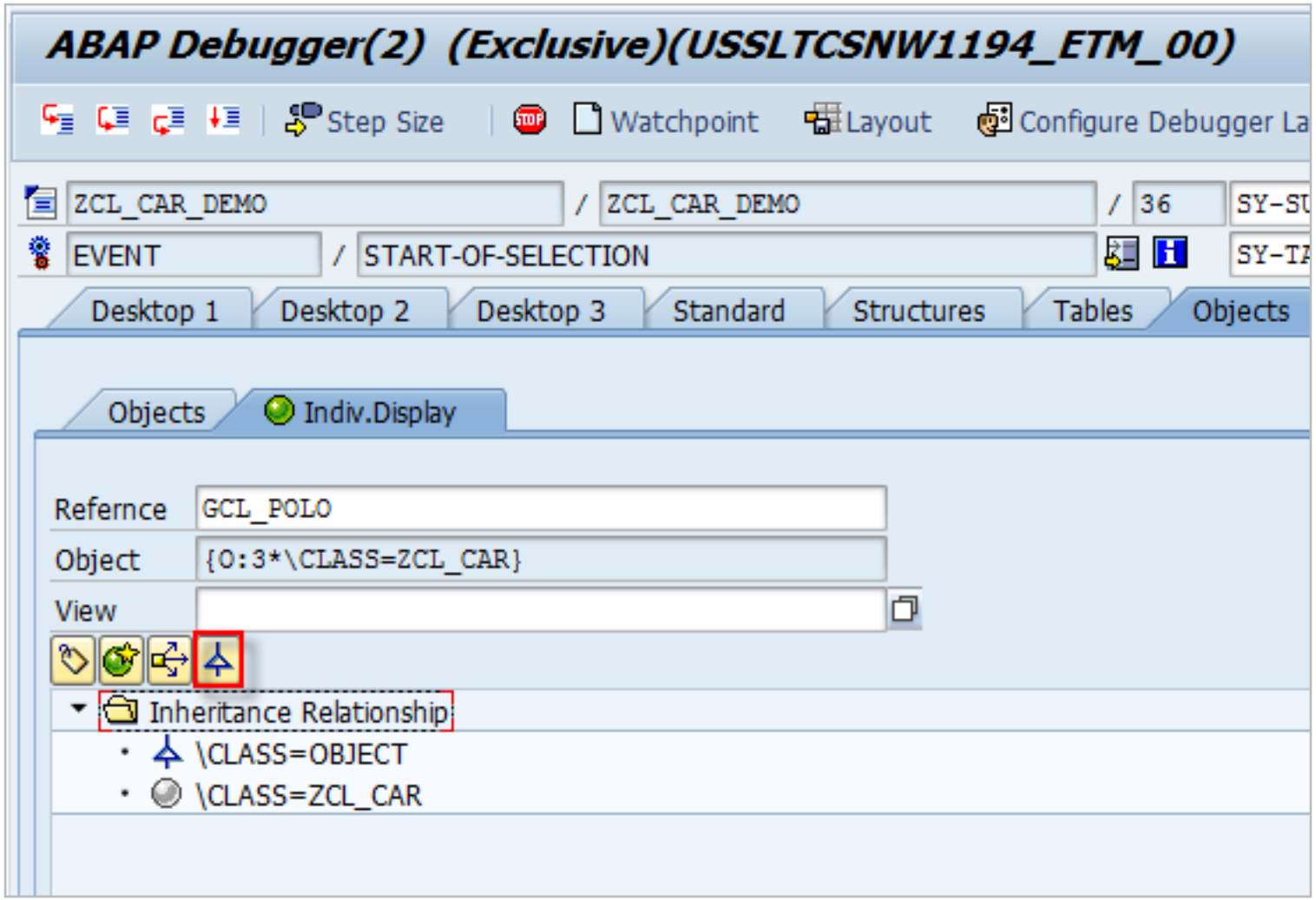
# Object Tab in Debugger Session






Copyright © 2022 Deloitte Development LLC. All rights reserved.

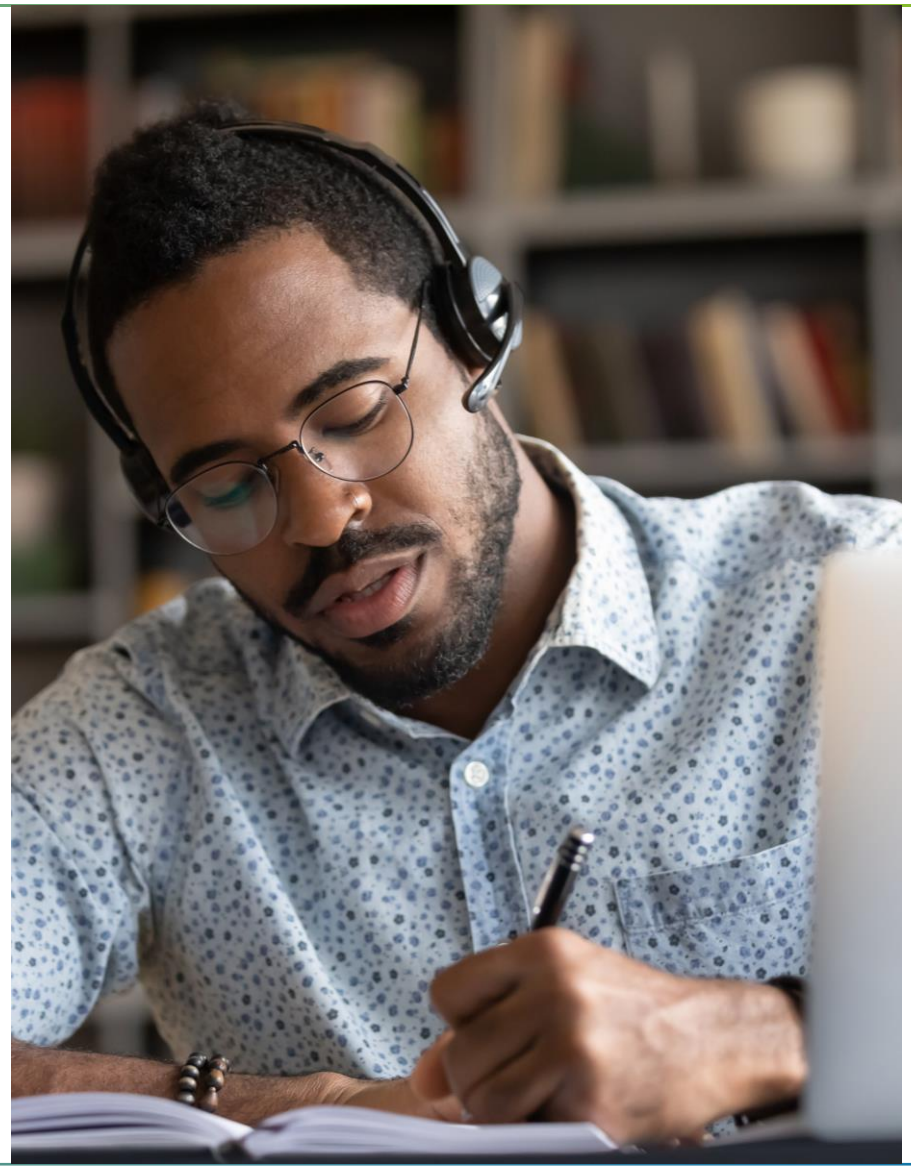
# Checking the Class Reference in Debugging






Copyright © 2022 Deloitte Development LLC. All rights reserved.

# Hands-On Labs



# Instructions

- The naming conventions should be followed.
- NN should be your unique 2-digit number.
- Assign all development objects to your Package.

**Create a global class (ZCLXX_SALES_ORDERS) to display sales order details with following characteristics:**

- Class Attributes—Sales Order No (Static Attribute).
- Constructor—Perform Authorization Check [Pass VA03 (Sales Order display) to the Function Module ‘authority_check_tcode’]

Copyright © 2022 Deloitte Development LLC. All rights reserved.

# Hands-On Activity 1

| Activity Details | |
|---|---|
| Problem Statement | **Get Header details (GET_HEADER_DETAILS) from sales master (VBAK table)** |
| Output | Ship to Party, Sold to Party, Order Type, Sales Area [concatenation of Sale Org, Division and Distribution channel], Document Date. (Use table VBAK and get Ship to from VBPA mapping sales order number and PARVW (Partner function) = ‘WE’ (Ship to). |



Copyright © 2022 Deloitte Development LLC. All rights reserved.

# Hands-On Activity 2

| Activity Details | |
|---|---|
| Problem Statement | **Get Item Details (GET_ITEM_DETAILS) from VBAP table** |
| Output | Item Category, Material, Quantity, Net Value, Pricing Date (Get the Pricing date from VBKD table) |



Copyright © 2022 Deloitte Development LLC. All rights reserved.

# Hands-On Activity 3

| Activity Details | |
|---|---|
| Problem Statement | **Display Sales Order Header (HEADER_DISPLAY)** |
| Output | Write the above attributes in the format below. |

| | |
|---|---|
| **Sales Order Number** | **<Order No.>** |
| **Ship to Party** | <Ship To Party> |
| **Sold to Party** | <Sold To Party> |
| **Order Type** | <Order Type> |
| **Sales Area** | <Sales Area> |
| **Document Date** | <Doc Date> |

Copyright © 2022 Deloitte Development LLC. All rights reserved.

# Hands-On Activity 4

| Activity Details | |
|---|---|
| Problem Statement | **Display Sales Order Items (ITEMS_DISPLAY)** |
| Output | Write the above attributes in the format below. (ALV Display to be used) |

***Sales Order Details***

| Item | Material | Quantity | Net Value | Pricing Date |
|---|---|---|---|---|
| TAN | M-01 | 1,000 | 1.680,00 | 21.01.1997 |
| TAN | M-02 | 2,000 | 3.836,00 | 21.01.1997 |
| TAN | M-03 | 1,000 | 1.848,00 | 21.01.1997 |
| TAN | M-04 | 1,000 | 1.988,00 | 21.01.1997 |



Copyright © 2022 Deloitte Development LLC. All rights reserved.

# Hands-On Activity 5

| Activity Details | |
|---|---|
| Problem Statement | **Create a custom program to get Sales Order No from selection screen, with a radio button to select either 'Header Details' or 'Item Details' and print those attributes on list output.** |



Copyright © 2022 Deloitte Development LLC. All rights reserved.

# Hands-On Activity 6

| Activity Details: Create an executable program to demonstrate interfaces | |
|---|---|
| Problem Statement | • Create an interface DOCUMENT with a method DISPLAY.<br>• Create a class INVOICE with an attribute 'INVOICE_ID' (type integer). The invoice id should be incremented every time an instance of the class is created. Implement the interface method DISPLAY to display the value of INVOICE_ID.<br>• Similarly create a class TEXTDOC with an attribute 'TEXT_MATTER' (type string). The default value of TEXT_MATTER is 'Hello OOABAP'. Implement the interface method DISPLAY to display the value of TEXT_MATTER.<br>• In the program, create instances of the type, INVOICE and TEXTDOC, and call respective display methods, and display the results. |



Copyright © 2022 Deloitte Development LLC. All rights reserved.

# Hands-On Activity 7

| Activity Details: Create an executable program to demonstrate Inheritance and Redefinition | |
|---|---|
| Problem Statement | • Create class CL_AIRPLANE with attributes name (name of the airplane) and no of airplanes (count of airplanes). Attribute name should be inheritable.<br>• Add methods DISPLAY_ATTRIBUTES (displays the name of the plane) and DISPLAY_AIRPLANE_COUNT which returns the total count of airplanes<br>• The count of airplanes should be incremented every time an instance of the class is created.<br>• Create a class CL_CARGOPLANE inheriting from CL_AIRPLANE with attributes for capacity of cargo plane (Table: SCPLANE, FIELD: CARGOMAX) and count of cargo plane.<br>• The method DISPLAY_ATTRIBTUES should display the name of the plane and cargo capacity.<br>• Add method DISPLAY_CARGO_PLANE_COUNT which returns the count of cargo planes<br>• The count of cargo planes should be incremented every time an instance of the class is created.<br>• Create a class CL_PASSENGERPLANE inheriting from CL_AIRPLANE with attributes for passenger capacity and count of passenger plane. |



Copyright © 2022 Deloitte Development LLC. All rights reserved.

# Hands-On Activity 7 (Cont.)

| Activity Details: Create an executable program to demonstrate Inheritance and Redefinition | |
|---|---|
| Problem Statement | • The method DISPLAY_ATTRIBTUES should display the name of the plane and passenger capacity.<br>• Add method DISPLAY_PASSGR_PLANE_COUNT which returns the count of passenger planes.<br>• The count of passenger planes should be incremented every time an instance of the class is created.<br>• In the program, create instances of the type of passenger plane (name: Boeing, seat count: 200) and cargo plane (name: Cargo, capacity: 100). Display the attributes and count of airplanes, passenger planes and cargo planes separately. |



Copyright © 2022 Deloitte Development LLC. All rights reserved.

# Summary

Here are the key learning points of the module.

- Object-Oriented Programming (OOP) is a programming methodology using objects consisting of data and methods.
- Classes have the following components: Attributes, Methods, Events, and Interfaces.
- A Super class is specified to define handlers for all the subordinate classes of the exception classes.



Copyright © 2022 Deloitte Development LLC. All rights reserved.

Thank You

Deloitte.

**About Deloitte**
Deloitte refers to one or more of Deloitte Touche Tohmatsu Limited, a UK private company limited by guarantee ("DTTL"), its network of member firms, and their related entities. DTTL and each of its member firms are legally separate and independent entities. DTTL (also referred to as "Deloitte Global") does not provide services to clients. In the United States, Deloitte refers to one or more of the US member firms of DTTL, their related entities that operate using the "Deloitte" name in the United States and their respective affiliates. Certain services may not be available to attest clients under the rules and regulations of public accounting. Please see www.deloitte.com/about to learn more about our global network of member firms.

This communication contains general information only, and none of Deloitte Touche Tohmatsu Limited ("DTTL"), its global network of member firms or their related entities (collectively, the "Deloitte organization") is, by means of this communication, rendering professional advice or services. Before making any decision or taking any action that may affect your finances or your business, you should consult a qualified professional adviser.

No representations, warranties or undertakings (express or implied) are given as to the accuracy or completeness of the information in this communication, and none of DTTL, its member firms, related entities, employees or agents shall be liable or responsible for any loss or damage whatsoever arising directly or indirectly in connection with any person relying on this communication. DTTL and each of its member firms, and their related entities, are legally separate and independent entities.

Copyright © 2022 Deloitte Development LLC. All rights reserved.